Cairo University

Faculty of Computers and Artificial Intelligence

Assignment: Vole Machine Simulator

Course: CS213-Object Oriented Programming

Semester / Year: 1st semester 2025-2026

Instructor:Dr. Mohammad El-Ramly

Section Number: S23/S24
Team Leader: Mohammed Al-Sofi | **Phone:** 01105717126
Date: 2026/11/10

Team Members

Name	ID	Email
Hatem Ahmed	20253050	20253050@stud.fci-cu.edu.eg
Alaa Al-Deen Nabil	20253072	20253072@stud.fci-cu.edu.eg
Mohammed Mazen	20253073	20253073@stud.fci-cu.edu.eg
Mohammed Abdulmalik	20253081	20253081@stud.fci-cu.edu.eg

WORK BREAKDOWN TABLE

Member	Part 1	Implementation (Files & classes)
Hatem Ahmed	Discussed the Initial design with **ChatGPT**	CLI classes (`ConsoleUI`, `ConsoleView`) and `ProgramLoader`
Alaa Al-Deen Nabil	Edited the video and produced its introduction	`Cpu`, `Memory`, `RegisterFile`; owner of the GitHub repository
Mohammed Mazen	Discussed the Initial design with **Gemini**	All header files (class interfaces); `Instruction` and its subclasses (except the ALU instructions), `InstructionDecoder`, `Screen`, `Machine`, `main.cpp`
Mohammed Abdulmalik	Discussed the Initial design with **Claude**	ALU instructions and `Alu`
Whole Team	The initial design was discussed and developed by the whole team together	

Part 3: GUI (AI-assisted)

The GUI was split into independent parts. Each part was generated with AI assistance and pushed by the members listed below. Mohammed Mazen defined the split, integrated the parts, and built and ran the application.

Member	***GUI Files (`.h` and `.cpp`)***
Hatem Ahmed	*`ProgramListPanel`, `StatusText`, `ThemeManager`*
Alaa Al-Deen Nabil	*`MainWindow`, `MachineSnapshot`, `InstructionSetDialog`*
Mohammed Mazen	*`MemoryPanel`, `ScreenPanel`, `RegisterPanel`*
Mohammed Abdulmalik	*`ControlBar`, `GuiTypes`, `InstructionDescriber`*

1. INTRODUCTION

1.1 What the Vole machine is

The Vole machine is a hypothetical computer introduced by J. Glenn Brookshear in "Computer Science: An Overview" (11th edition) to explain how a real CPU executes a program stored in memory. Although it is very small, it contains all the essential parts of a real computer: main memory, general-purpose registers, a program counter, an instruction register, an arithmetic/logic unit, and its own machine language.
A Vole program is a sequence of machine instructions stored in memory. The machine runs the program by repeating a simple cycle: fetch the next instruction, decode it, and execute it.
The goal of this assignment is to design (Part 1) and then implement (Parts 2 and 3) an object-oriented simulator of this machine. The simulator lets a user load a program from a file, run it all at once or step by step, and watch the state of the machine (memory, registers, PC, IR and the screen). In this part of the work we model the machine as a set of classes and relationships in a UML class diagram, with the help of three different AI tools, and we refine the best result into a final design.

1.2 Architecture: main memory, general registers, PC, IR

The machine consists of the following components:

- **Main memory:** 256 cells. Each cell holds 8 bits (one byte). The cells are addressed with two hexadecimal digits, from 00 to FF.
- **General-purpose registers:** 16 registers numbered 0 to F, each holding 8 bits. They hold the values the program is working on. Register 0 also has a special role in the JUMP instruction.
- **Program Counter (PC):** An 8-bit register holding the ADDRESS of the next instruction to be fetched. It does not hold the instruction itself.
- **Instruction Register (IR):** A 16-bit register holding the CURRENT instruction, the one that has been fetched and is being decoded and executed.
- **Control Unit (CU):** Coordinates the machine cycle: it fetches the instruction, decodes it, and makes sure it is executed.
- **Arithmetic/Logic Unit (ALU):** Performs the calculations: integer addition, floating-point addition, OR, AND, XOR, and rotation.
- **Screen:** The simulator's output device. In our version of the machine the memory address

FF is mapped to the screen: storing a value to address FF displays it on the screen instead of keeping it in memory.

Data representation:

- All values in programs and in the simulator display are written in hexadecimal. One byte is two hex digits (for example, A3 = 1010 0011).
- Integers are stored in two's complement, so one byte represents values from -128 to +127 (80 to 7F in hex).
- Fractional values use an 8-bit floating-point format: 1 sign bit, 3 exponent bits (stored in excess-four notation) and 4 mantissa bits. For example, 5C = 0 101 1100 has exponent 101 = 5, so the real exponent is 5 - 4 = 1, and the mantissa .1100 = 0.75. The value is 0.75 x 2^1 = 1.5.

1.3 Instruction format and the instruction set (12 instructions)

Every instruction is 16 bits long, which is four hexadecimal digits:

[op-code] [digit 2] [digit 3] [digit 4]

- The first digit is the **OP-CODE**. It says which operation to perform.
- The remaining three digits are the **OPERANDS**. Their meaning depends on the op-code.

An instruction occupies TWO consecutive memory cells: the cell at address A holds the first two digits and the cell at address A+1 holds the last two digits. Therefore, instructions are placed at even addresses, and this is why a program must be loaded at an even address.

Notation used in the instruction table:

- **R, S, T:** a register number (one hex digit, 0 to F)
- **XY:** a two-digit hex number. It is either a memory address or an immediate value, depending on the instruction.
- **0:** a digit that must be zero (it is not used)

Op	Format	Meaning
1	RXY	LOAD register R with the contents of memory cell XY. (Example: 14A3 puts the

Op	Format	Meaning
		contents of cell A3 in register 4).
2	RXY	LOAD register R with the value XY itself. (Example: 20A3 puts the value A3 in register 0).
3	RXY	STORE the contents of register R in memory cell XY. Special case: if XY = FF, the value is written to the screen. (Example: 35B1 stores register 5 in cell B1. 32FF displays register 2 on the screen).
4	0RS	MOVE (copy) the contents of register R to register S. (Example: 40A4 copies register A into register 4).
5	RST	ADD registers S and T as two's complement integers and put the result in register R. (Example: 5726 puts (R2 + R6) in R7).
6	RST	ADD registers S and T as floating-point values and put the floating-point result in register R. (Example: 634E puts (R4 + RE) in R3).
7	RST	OR the bit patterns of registers S and T, result in R. (Example: 7CB4 puts (RB OR R4) in RC).
8	RST	AND the bit patterns of registers S and T, result in R. (Example: 8045 puts (R4 AND R5) in R0).
9	RST	XOR the bit patterns of

Op	Format	Meaning
		registers S and T, result in R. (Example: 95F3 puts (RF XOR R3) in R5).
A	R0X	ROTATE the bit pattern in register R to the right X times. (Example: A403 rotates R4 right by 3 bits).
B	RXY	JUMP to the instruction at address XY if the contents of register R are equal to the contents of register 0. (Example: B43C jumps to address 3C if R4 equals R0).
C	000	HALT: stop the execution of the program. (Example: C000).

Notes on the instruction set:

- Instruction 3 behaves differently when XY = FF, because that address is mapped to the screen. This is an extension of the original Brookshear machine used in this assignment.
- Instruction B with R = 0 compares register 0 with itself, so the condition is always true. It therefore works as an unconditional jump.
- Instructions 4, A and C contain digits that must be zero. An instruction that violates this is invalid.
- Only these 12 instructions are supported by the simulator.

Invalid instructions: An instruction is invalid if (a) its op-code is not one of 1 to C (that is, the op-code is 0, D, E or F), or (b) it breaks the format of its op-code, for example when a digit that must be zero is not zero (as in 4100, A410 or C001). The simulator must reject invalid instructions, and must reject any program that contains them.

1.4 The machine cycle: Fetch - Decode - Execute

Before the program starts, it is loaded into memory at an even address and the PC is set to the address of its first instruction. Then the machine repeats three steps until it executes a HALT instruction.

1. **FETCH:** The two memory cells at the address held in the PC are read and placed in the IR (16 bits). Then the PC is increased by 2 so that it points to the next instruction.

2. **DECODE:** The IR is split into the op-code and the operands, and the instruction is checked to be valid.
3. **EXECUTE:** The operation is performed: data is moved, the ALU calculates, a value is stored, or the PC is changed by a JUMP. After this, the cycle returns to the fetch step.

Because the PC has already been increased during the fetch step, a JUMP replaces the PC value with a new address, and the next fetch starts from there.

1.5 Program file format

Programs are text files with these rules:

- Each instruction is written on its own line as exactly four hex digits.
- Anything after the four digits on a line is ignored (so a comment may follow the instruction).
- A line that starts with a semicolon (;) is a comment and is ignored.
- Empty lines are ignored.

1.6 What the simulator must do (the 11 requirements)

1. Offers a menu of choices.
2. Loads a program from a file into memory starting at an even address chosen by the user.
3. Lets the user overwrite the content of any register or memory cell with a new value.
4. Sets the PC to the start address of the program.
5. Displays the memory as a matrix, the IR, the PC and the 16 registers.
6. Fetches instructions one by one into the IR, validates them, decodes them, executes them, and updates the PC.
7. Lets the user choose between running the program at once or step by step.
8. For a run at once: displays the memory, PC, IR, the 16 registers and the screen output after the program finishes.
9. For step-by-step: displays the same information after every step and waits for the user to press Enter before the next step.
10. Asks the user whether to exit or to load another program.
11. Rejects invalid instructions, and rejects programs that contain invalid instructions.

2. INITIAL DESIGN

2.1 Approach and guiding principles

1.1 Goal

Design a simulator for the Vole machine that covers all the functions required by the assignment: a menu, loading a program from a file into an even memory address, editing registers and memory cells, setting the PC, displaying the machine state, the Fetch/Decode/Execute cycle, running the whole program or step by step, displaying the screen, rejecting invalid instructions, and asking the user whether to exit or load another program.

1.2 Guiding principle: a design model, not a copy of reality

A class represents a RESPONSIBILITY, not necessarily a physical part of the machine. For example, in a real machine an instruction is just data, but we give it behavior (execute) because this organizes the code better. We also added classes that do not exist in the real hardware (ProgramLoader, InstructionDecoder, Menu) because each has a clear responsibility.
Reason: the goal of the assignment is to learn OOP modeling and design, so the quality criterion is how easy the design is to understand, modify and test, not how closely it mirrors the hardware.

1.3 Criteria used to judge each class

- Its responsibility can be described in one sentence (Single Responsibility).
- Changing something requires modifying only one place (Low Coupling).
- Its existence can be justified.

1.4 Working style: incremental versions

We did not aim for a perfect design from the start. We developed the design in versions and deliberately left some points open. The target was a COMPLETE design (every required function has a place), not a perfect one.

2.2 Classes and their responsibilities

Class Name	Responsibilities
Machine	Main coordinator; owns and instantiates all system components.
Menu	Handles user interaction, main application loop, and user commands.

Class Name	Responsibilities
Screen	Displays the machine state (memory, registers, PC, IR) and program output.
ProgramLoader	Reads program files, validates instructions, and loads them into memory.
CPU	Acts as a container for CPU components and a facade for execution state.
CU	Control Unit; manages the fetch-decode-execute machine cycle.
ALU	Performs mathematical and logical calculations for instructions.
Memory	Stores data and instructions in a 256-cell array.
Register	Base class for holding bit patterns with specific sizes.
ProgramCounter	Holds the address of the next instruction to execute.
InstructionRegister	Holds the currently fetched instruction and provides its opcode.
Instruction	Abstract base for instruction execution and validation logic.
InstructionDecoder	Factory class to interpret opcodes and instantiate the correct Instruction object.

2.3 Attributes and methods of each class

- **Machine:** *Attributes:* cpu, memory, screen, loader, decoder, menu. *Methods:* start().
- **Menu:** *Methods:* mainLoop(), showMainMenu(), loadProgram(), runAll(), runStep(), editRegister(), editMemory(), showState(), askExitOrLoad().
- **Screen:** *Attributes:* output. *Methods:* showMemory(), showRegisters(), showPCAndIR(), appendOutput(), showOutput(), clearOutput().
- **ProgramLoader:** *Attributes:* lastError. *Methods:* load(), getLastError(), isEvenAddress(), cleanLine().

- **CPU:** *Attributes:* registers[16], pc, ir, alu, cu, halted. *Methods:* getRegister(), getPC(), getIR(), getMemory(), getScreen(), halt(), isHalted().
- **CU:** *Methods:* step(), runAll(), fetch().
- **ALU:** *Methods:* addInt(), addFloat(), bitOr(), bitAnd(), bitXor(), rotateRight().
- **Memory:** *Attributes:* cells[256]. *Methods:* read(), write().
- **Register:** *Attributes:* value, sizeInBits. *Methods:* getValue(), setValue().
- **ProgramCounter:** *Methods:* increment().
- **InstructionRegister:** *Methods:* getOpcode().
- **Instruction:** *Attributes:* opcode, n1, n2, n3. *Methods:* xy(), isValid(), execute().
- **InstructionDecoder:** *Methods:* decode().
- **12 Specific Instructions:** *Methods:* isValid(), execute(cpu).

2.4 Relationships between classes

- **Composition:** Machine -> its components; CPU -> 16 registers, PC, IR, ALU, CU.
  The whole creates and destroys its parts, and they have no meaning without it.
- **Association:** CPU -> Memory, CPU -> Screen, CU -> CPU, Menu -> Machine.
  A permanent reference without ownership. The owner of Memory and Screen is Machine.
- **Dependency:** ProgramLoader -> Memory, Instruction -> CPU, Screen -> Memory.
  Used only as a parameter or local variable.
- **Inheritance:** Register -> ProgramCounter, InstructionRegister; Instruction -> the 12 instruction classes.
  A genuine is-a relationship.

2.5 PlantUML code

```
@startuml
title Vole Machine - Initial Design v1

enum StepStatus {
  OK
  HALTED
  INVALID_INSTRUCTION
}

class Machine {
  - cpu : CPU
  - memory : Memory
```

```
 - screen : Screen
 - loader : ProgramLoader
 - decoder : InstructionDecoder
 - menu : Menu
 + start()
}

class Menu {
 + mainLoop()
 + showMainMenu()
 + loadProgram()
 + runAll()
 + runStep()
 + editRegister()
 + editMemory()
 + showState()
 + askExitOrLoad()
}

class Screen {
 - output : string
 + showMemory(m : Memory)
 + showRegisters(c : CPU)
 + showPCAndIR(c : CPU)
 + appendOutput(value : int)
 + showOutput()
 + clearOutput()
}

class ProgramLoader {
 - lastError : string
 + load(file : string, start : int, m : Memory) : bool
 + getLastError() : string
 - isEvenAddress(a : int) : bool
 - cleanLine(line : string) : string
}

class CPU {
 - registers : Register[16]
```

```
 - pc : ProgramCounter
 - ir : InstructionRegister
 - alu : ALU
 - cu : CU
 - halted : bool
 + getRegister(i : int) : Register
 + getPC() : ProgramCounter
 + getIR() : InstructionRegister
 + getMemory() : Memory
 + getScreen() : Screen
 + halt()
 + isHalted() : bool
}

class CU {
 + step() : StepStatus
 + runAll() : StepStatus
 - fetch()
}

class ALU {
 + addInt(a : int, b : int) : int
 + addFloat(a : int, b : int) : int
 + bitOr(a : int, b : int) : int
 + bitAnd(a : int, b : int) : int
 + bitXor(a : int, b : int) : int
 + rotateRight(v : int, n : int) : int
}

class Memory {
 - cells : int[256]
 + read(address : int) : int
 + write(address : int, value : int)
}

class Register {
 # value : int
 # sizeInBits : int
 + getValue() : int
```

```
  + setValue(v : int)
}
class ProgramCounter {
  + increment(n : int)
}
class InstructionRegister {
  + getOpcode() : int
}
Register <|-- ProgramCounter
Register <|-- InstructionRegister

abstract class Instruction {
  # opcode : int
  # n1 : int
  # n2 : int
  # n3 : int
  # xy() : int
  + {abstract} isValid() : bool
  + {abstract} execute(cpu : CPU)
}

class InstructionDecoder {
  + decode(code : int) : Instruction
}

class LoadMemInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class LoadImmInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class StoreInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class MoveInstruction <> {
  + isValid() : bool
```

```
  + execute(cpu : CPU)
}
class AddIntInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class AddFloatInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class OrInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class AndInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class XorInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class RotateInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class JumpInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}
class HaltInstruction <> {
  + isValid() : bool
  + execute(cpu : CPU)
}

Instruction <|-- LoadMemInstruction
Instruction <|-- LoadImmInstruction
Instruction <|-- StoreInstruction
Instruction <|-- MoveInstruction
```

```
Instruction <|-- AddIntInstruction
Instruction <|-- AddFloatInstruction
Instruction <|-- OrInstruction
Instruction <|-- AndInstruction
Instruction <|-- XorInstruction
Instruction <|-- RotateInstruction
Instruction <|-- JumpInstruction
Instruction <|-- HaltInstruction

note right of JumpInstruction
  Jump if reg[R] == reg[0]
  then PC = XY (XY must be even)
end note
note right of StoreInstruction
  If XY == FF the value goes
  to Screen, not to Memory
end note
note right of InstructionDecoder
  opcodes 0, D, E, F
  are invalid (returns null)
end note

' ---- Composition ----
Machine *-- CPU
Machine *-- Memory
Machine *-- Screen
Machine *-- ProgramLoader
Machine *-- InstructionDecoder
Machine *-- Menu
CPU "1" *-- "16" Register
CPU *-- ProgramCounter
CPU *-- InstructionRegister
CPU *-- ALU
CPU *-- CU

' ---- Association ----
Menu --> Machine : commands
CPU --> Memory : reads/writes
CPU --> Screen : output at FF
```

```
CU --> CPU : controls
CU --> InstructionDecoder : uses

' ---- Dependency ----
Menu ..> StepStatus
CU ..> StepStatus : returns
CU ..> Instruction : runs
ProgramLoader ..> Memory : writes
ProgramLoader ..> InstructionDecoder : validates
Screen ..> Memory : displays
Screen ..> CPU : displays
InstructionDecoder ..> Instruction : creates
Instruction ..> CPU : executes on
AddIntInstruction ..> ALU : uses
AddFloatInstruction ..> ALU : uses
OrInstruction ..> ALU : uses
AndInstruction ..> ALU : uses
XorInstruction ..> ALU : uses
RotateInstruction ..> ALU : uses
@enduml
```

2.6 UML class diagram

Diagram Link: Initial Design Diagram (SVG)
[Note: Since the diagram is too large, the link is provided above. You can also manually insert the image file here if desired.]

2.7 Design rationale (decisions and reasons)

Decision 1: Machine as the coordinator
Problem: Who assembles the parts and runs them together?
Decision: Machine owns (Composition) the CPU, Memory, Screen, ProgramLoader, InstructionDecoder and Menu.
Reason: One single point creates the components and connects them, so no component needs to create the others. Memory and Screen have one owner, and other classes access them through references.

Decision 2: Separating Menu from Screen

Problem: Where do we put user interaction, and where do we put display?
Decision: Menu receives the user's choices and contains the main loop (mainLoop). Screen contains all the display logic (memory as a matrix, IR, PC, registers, and program output). The menu gives the command and the screen does the displaying. Menu also offers editRegister and editMemory.
Reason: Separates INPUT from OUTPUT. If the display changes (for example in a later GUI version), the menu logic is not touched. The two edit options cover requirement 3 of the assignment.
Note: Placing the main loop in Menu is a preliminary decision that may be revisited.

Decision 3: Register as the parent of ProgramCounter and InstructionRegister
Problem: PC, IR and the general registers all store a bit pattern.
Decision: Register is the parent class. ProgramCounter inherits it and adds increment. InstructionRegister inherits it and adds getOpcode.
Reason: A genuine is-a relationship, and the additions in the two children justify the inheritance.
Resulting problem: General registers and the PC are 8 bits, while the IR is 16 bits.
Solution: A sizeInBits attribute in the parent restricts the value in setValue.

Decision 4: Memory with 256 cells
Decision: Memory contains cells[256] and only read and write.
Reason: A single responsibility: storage. It knows nothing about instructions or the screen.
Note: Writing to address FF does not reach memory but the screen (see Decision 10).

Decision 5: Instruction hierarchy (Inheritance + Polymorphism)
Problem: There are 12 kinds of instructions, each with different logic.
Options: (a) One huge switch in a single place. (b) One class per instruction inheriting from an abstract parent.
Decision: Option (b): abstract Instruction with 12 children. Each child has isValid and execute.
Reason: Adding a new instruction does not change existing code (Open/Closed Principle). Each instruction carries its own logic and validity rules. Each instruction can be tested on its own.

Decision 6: InstructionDecoder (Factory pattern)
Problem: Who converts the code 2005 into a LoadImmInstruction object?
Decision: A separate class, InstructionDecoder.decode(code), reads the opcode and creates the right object. It returns null for unknown opcodes (0, D, E, F).
Reason: Separates the responsibility of RECOGNIZING and CREATING an instruction from the responsibility of EXECUTING it.

Decision 7: Instructions execute themselves; CU manages the cycle

Problem: Should execution be inside the instruction or inside the CU?
Options: (a) The instruction executes everything and the CU is removed. (b) The CU executes, and the instruction is only data. (c) Hybrid.
Decision: Hybrid. Each instruction implements execute(cpu). The CU manages the cycle (step, runAll, fetch).
Reason: (a) removes the role of the CU and makes the CPU too big. (b) brings back the huge switch and removes the benefit of polymorphism. The hybrid keeps a real role for each.
Cost: An instruction needs access to many components, so it receives the CPU as a parameter to avoid holding many references.

Decision 8: Separate ProgramLoader
Decision: A class responsible for reading the file (ignoring comments and empty lines), checking that the start address is even, validating the instructions, and writing them to Memory.
Reason: Keeps file reading and validation out of Machine, so it does not become too big, and makes loading testable on its own.

Decision 9: Rejecting invalid programs at load time
Problem: When do we reject a program that contains invalid instructions?
Options: At execution time (to avoid duplicated logic), or at load time.
Decision: At LOAD time, keeping a backup check at execution time.
Reason: (1) The assignment says the simulator should reject programs with invalid instructions. Rejecting after half of the program has run is not a rejection. (2) There is no real duplication, because isValid() is separate from execute(), so the loader can call it without executing anything. (3) The execution-time check remains because the user may edit memory manually, or a JUMP may land on data.

Decision 10: Reporting rejection, and the screen output
Reporting: Return values. load() returns bool, with getLastError() giving the line number and reason. step() returns an enum StepStatus (OK, HALTED, INVALID_INSTRUCTION).
Reason: Simpler than exceptions at this stage and enough for a project of this size.
Screen: Screen.output stores what was written to the screen and it is displayed after the program finishes (or after each step), as the assignment requires.
FF: StoreInstruction checks whether XY == FF and sends the value to the Screen instead of Memory, so Memory stays simple and does not know the screen.
Note: Preliminary decision: the value is written to the screen only, not to cell FF.

Decision 11: ALU with separate functions
Decision: ALU contains addInt, addFloat, bitOr, bitAnd, bitXor and rotateRight. The floating-point logic is a separate function inside ALU.
Reason: ALU gathers the arithmetic and logic operations so instructions do not repeat them.

Interpreting a register as floating point (sign + exponent + mantissa) is very different from integer addition, so it deserves its own function.

Decision 12: Progress of execution is controlled by the CU
Problem: Who controls how execution advances (run all / one step): the CU or the Menu?
Decision: Preliminarily the CU (step and runAll). The Menu only requests them and then asks the Screen to display.
Reason: Keeps the Fetch/Decode/Execute loop in one place.
Note: This needs further thought and may change.

Mapping Assignment Requirements to Classes

Requirement	Assigned Class
Menu and main loop	Menu
Load from file to even address	ProgramLoader
Edit a register or a cell	Menu (calls Register / Memory)
Set PC to program start	Machine (via ProgramCounter)
Display memory, IR, PC, registers	Screen
Fetch	CU
Decode and validation	InstructionDecoder and Instruction.isValid()
Execute	Instruction.execute()
Run all / Step	CU (requested by Menu)
Screen display	Screen
Reject invalid instructions	ProgramLoader at load time, CU as a backup
Ask to exit or load another	Menu

2.8 Version history and open points

Version History

- **v0.1:** First list of classes: Machine, Screen, Menu, Instruction, Decoder, Memory,

Register, ALU, CU, CPU.

- **v0.2:** Added ProgramLoader; edit register/memory options in Menu; display split between Menu and Screen; hybrid execution model; load/store moved from CU into the instructions; execute(cpu) added.
- **v0.5:** (v0.3/0.4 was skipped because the changes were large.) Rejection at load time; return values for reporting; StepStatus; output kept in Screen.
- **v1:** Details of the 12 instructions (format of each and its validity rules); getMemory and getScreen added to CPU. First version that covers all required functions.

Notes on versioning:
- Only v0.5 and v1 were documented (committed) in the repository. The earlier versions (v0.1, v0.2, v0.4) were working drafts and are described here from our design discussions.
- Why we jumped from v0.5 to v1: v0.5 still had the details of the 12 instructions postponed. Once these details were added, every function required by the assignment had a responsible class and method, so the design became complete and integrated. We therefore considered it a full initial design and named it v1 instead of v0.6.
- During this phase we discussed with an AI the question of whether an instruction should execute itself or whether a switch should handle all instructions. However, we decided the initial design ourselves.

Open Points

The following points were debated inside the team and were left open deliberately, since we expected them to be discussed with the AIs:

1. Distribution of responsibilities between CPU and CU. The CPU may become only a container. Options include moving register access to the CU, or making the CPU a facade with methods such as writeMemory that decide whether FF goes to the screen.
2. Who controls the progress of execution: CU or Menu.
3. The circular reference between CPU and CU.
4. The display format of the screen (hex, decimal or character).
5. Whether a write to FF goes to the screen only or to cell FF too.
6. Repeated steps among instructions 5-9 (read S and T, compute, write R).
7. Additional checks such as a program exceeding memory bounds at load time.

Note: this list does NOT limit future development to these points only. These are the points that were discussed within the team and that we took into account as likely subjects of discussion with the AIs. Any other part of the design may also be improved.

3. AI DESIGNS

3.1 AI #1: Gemini

3.1.1 Tool Information

- **Tool:** Google Gemini (chat interface)
- **Model/version:** not shown in the app!
- **Date of use:** 2026/10/25
- **Input given:** The unified prompt (Vole machine description, the 11 requirements, program file format) + our initial PlantUML design (Section 2.5).
- **Not given:** The original assignment file.
- **Number of rounds:** 6 (1 design request + 5 follow-up rounds of feedback/questions)

3.1.2 Conversation Summary

The full conversation is available at the following link:
https://share.gemini.google/JaChcq6KSI8m
The conversation went through six rounds. Our team used the unified prompt and then reviewed each reply critically before asking for the next improvement.
Round 0 - Unified prompt and first design.
We gave Gemini the description of the Vole machine, the 11 requirements, the program file format, and our initial PlantUML design (Section 2.5). We asked it to review our design, produce its own design with PlantUML code, and justify its decisions. Gemini praised our use of an abstract Instruction class with 12 subclasses, but criticised two things: the ProgramCounter and InstructionRegister inheriting from Register (the IR is 16-bit while the other registers are 8-bit), and the CPU acting as a "god object" that every instruction reaches through. It proposed a refined design with a separate ControlUnit, explicit references (CPU&, Memory&, Screen&) passed to execute(), and the ProgramLoader using the InstructionDecoder to reject invalid instructions.
Round 1 - Our feedback on Gemini's design.
We found four problems in its design:

1. A chicken-and-egg problem: the decoder had to create an instruction object before it could check whether it was valid, because validation lived inside the Instruction classes. We proposed moving validation out into a separate validator that works on the raw code.
2. Run-all and step-by-step control should belong to the Menu, not to the ControlUnit, which

should only perform the machine cycle.

3. The Screen should be independent of the Machine and should only act as an output device.
4. The Menu should not be part of the Machine (no composition) and should be an independent interface class.

Gemini accepted all four points. It added the InstructionValidator class, removed runAll() and isValid() from the ControlUnit and instructions, removed the Menu from the Machine composition, and removed state display from the Screen.

Round 2 - "Why does Instruction need to access Screen?"

We asked why instructions needed a Screen reference. Gemini admitted this was an architectural leak and redesigned the output as true memory-mapped I/O: Memory::write() intercepts address FF and forwards the value to the Screen. As a result the instruction signature became execute(CPU&, Memory&) and the instructions no longer know about the Screen.

Round 3 - Questions about reset(), halt() and the getters.

We asked what reset() and halt() exactly do, how the getters should protect encapsulation, and how the ControlUnit moves data between memory and registers. Gemini defined reset() as returning the CPU to its power-on state (registers, PC and IR cleared, halted flag cleared) and halt() as setting the halted flag. It recommended const getters for read-only inspection and explained that the ControlUnit works with registers through the CPU, so no separate RegisterFile class is needed.

Round 4 - The ControlUnit access asymmetry.

We asked why the ControlUnit accesses Memory directly but reaches the registers only through the CPU. Gemini explained that registers are internal CPU state while memory is an external resource, and admitted that the asymmetry is not ideal. It suggested making the CPU the only mediator for memory access (readMemory/writeMemory).

Round 5 - Request for the final version.

We asked for the best possible version without major redesign. Gemini produced the final design and PlantUML code (Section 3.2.3). It kept the existing structure and did not apply the CPU-mediated memory access suggested in Round 4, so the ControlUnit still holds a direct Memory reference in the final diagram.

3.1.3 Final PlantUML Code (Gemini)

```
@startuml
title Vole Machine - Final Architecture

skinparam classAttributeIconSize 0

enum StepStatus {
```

```
 OK
 HALTED
 INVALID_INSTRUCTION
}

class Machine {
 - cpu : CPU
 - memory : Memory
 - screen : Screen
 - loader : ProgramLoader
 - decoder : InstructionDecoder
 + start() : void
}

class Menu {
 - machine : Machine&
 + mainLoop() : void
 - showMainMenu() : void
 - handleLoadProgram() : void
 - handleEditRegister() : void
 - handleEditMemory() : void
 - handleRunAll() : void
 - handleRunStep() : void
 - displayState() : void
 - promptExitOrReload() : bool
}

class Screen {
 - outputBuffer : string
 + append(value : int) : void
 + showOutput() : void
 + clear() : void
}

class ProgramLoader {
 - lastError : string
 + load(filepath : string, startAddress : int, memory : Memory&) : bool
 + getLastError() : string
 - cleanLine(line : string) : string
```

```
}

class InstructionValidator {
  + {static} isValidInstruction(code : int) : bool
  + {static} isEvenAddress(address : int) : bool
}

class CPU {
  - registers : Register[16]
  - pc : ProgramCounter
  - ir : InstructionRegister
  - alu : ALU
  - cu : ControlUnit
  - halted : bool
  + getRegister(index : int) : Register&
  + setRegister(index : int, value : int) : void
  + getPC() : ProgramCounter&
  + getIR() : InstructionRegister&
  + getALU() : ALU&
  + getControlUnit() : ControlUnit&
  + isHalted() : bool
  + halt() : void
  + reset() : void
}

class ControlUnit {
  - cpu : CPU&
  - memory : Memory&
  - decoder : InstructionDecoder&
  + step() : StepStatus
  - fetch() : int
}

class ALU {
  + addInt(a : int, b : int) : int
  + addFloat(a : int, b : int) : int
  + bitOr(a : int, b : int) : int
  + bitAnd(a : int, b : int) : int
  + bitXor(a : int, b : int) : int
```

```
 + rotateRight(value : int, count : int) : int
}

class Memory {
 - cells : int[256]
 - screen : Screen*
 + read(address : int) : int
 + write(address : int, value : int) : void
 + clear() : void
}

class Register {
 # value : int
 + getValue() : int
 + setValue(v : int) : void
}

class ProgramCounter {
 # value : int
 + getValue() : int
 + setValue(v : int) : void
 + increment(n : int) : void
}

class InstructionRegister {
 # value : int
 + getValue() : int
 + setValue(v : int) : void
}

abstract class Instruction {
 # opcode : int
 # reg1 : int
 # reg2 : int
 # reg3 : int
 # addressOrImm : int
 + {abstract} execute(cpu : CPU&, memory : Memory&) : void
}
```

```
class InstructionDecoder {
 + decode(code : int) : Instruction*
}

class LoadMemInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class LoadImmInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class StoreInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class MoveInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class AddIntInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class AddFloatInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class OrInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class AndInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class XorInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class RotateInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class JumpInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
}
class HaltInstruction {
 + execute(cpu : CPU&, memory : Memory&) : void
```

```
}

Instruction <|-- LoadMemInstruction
Instruction <|-- LoadImmInstruction
Instruction <|-- StoreInstruction
Instruction <|-- MoveInstruction
Instruction <|-- AddIntInstruction
Instruction <|-- AddFloatInstruction
Instruction <|-- OrInstruction
Instruction <|-- AndInstruction
Instruction <|-- XorInstruction
Instruction <|-- RotateInstruction
Instruction <|-- JumpInstruction
Instruction <|-- HaltInstruction

Machine *-- CPU
Machine *-- Memory
Machine *-- Screen
Machine *-- ProgramLoader
Machine *-- InstructionDecoder

CPU "1" *-- "16" Register
CPU *-- ProgramCounter
CPU *-- InstructionRegister
CPU *-- ALU
CPU *-- ControlUnit

Menu --> Machine : controls
ProgramLoader --> InstructionValidator : uses
ControlUnit --> InstructionDecoder : uses
ControlUnit --> Instruction : executes
ProgramLoader --> Memory : writes
InstructionDecoder --> Instruction : creates
Memory --> Screen : maps output at FF
@enduml
```

3.1.4 UML Class Diagram (Gemini)

https://github.com/AlaaAl-DeenNabilSarhanAl-Maliki/oop_assignment2_Vole_Machine/blob/main/Gemini%20gg%20final.svg

3.2 AI #2: ChatGPT

3.2.1 Tool Information

- **Tool:** ChatGPT (OpenAI)
- **Model/version:** [FILL IN - as shown in the app, e.g. GPT-5]
- **Date of use:**2025/10/25
- **Input given:** The unified prompt (Vole machine description, the 11 requirements, program file format) + our initial PlantUML design (Section 2.5).
- **Not given:** The original assignment file.
- **Number of rounds:** 4 (Round 0 to Round 3)

3.2.2 Conversation Summary

The full conversation is available at the following link:
https://chatgpt.com/share/6adefd76-2524-83e8-a30a-90e7df1d4775

The conversation went through four rounds. We started with the team's initial UML design for the Vole machine simulator, reviewed its architecture, discussed the responsibility of instruction validation, and then updated the UML to enforce validation of the entire program before installation and execution.

Round 0 - Initial design review and proposed architecture.

We provided the initial UML design and the complete Vole machine requirements, asking for a critique, an improved object-oriented design, complete PlantUML code, and justifications for the design decisions. The review identified several weaknesses: the circular dependency between CPU and CU, the questionable inheritance relationship between Register and the special registers PC and IR, the Screen mixing output-device responsibilities with state display, and insufficiently defined program validation. The proposed design removed CU, introduced SimulatorApp and ConsoleUI, separated Screen output from state display, and retained polymorphic instruction classes. It also introduced ProgramImage to separate program validation from installation.

Round 1 - Who should be responsible for instruction validation?

We asked which class should validate instructions and whether the validation method in each Instruction subclass should be static so that instructions could be validated without creating objects. The response divided validation responsibilities among three components:

ProgramLoader validates the complete program, InstructionDecoder identifies and validates individual instructions, and each concrete Instruction subclass implements its own encoding-validation rules through a virtual isEncodingValid() method. It recommended keeping this method non-static to preserve polymorphism and avoid introducing a separate registry of static validation functions.

Round 2 - The entire program must be validated before execution.

We clarified that the program must be rejected if even one instruction is invalid: validation must finish before execution begins. The response agreed and specified an all-or-nothing workflow. ProgramLoader checks every instruction using InstructionDecoder and produces a ProgramImage only if all instructions are valid. If any instruction is invalid, the program is rejected and must not be installed or executed. It also explained that runtime validation remains necessary because the user can modify memory after loading.

Round 3 - Request for the updated UML.

We asked for the UML to be updated to reflect the all-or-nothing validation requirement. The revised design made ProgramLoader responsible for validating every instruction before creating ProgramImage, while InstructionDecoder creates a candidate instruction and invokes its virtual isEncodingValid() method. ProgramImage was given a private constructor, intended to be accessible only to ProgramLoader, to prevent arbitrary creation of unvalidated program images. SimulatorApp installs the program only after successful validation, and Machine validates fetched instructions again during execution. The updated PlantUML was provided in a text box after the download link did not work.

Main outcome

The central design decision reached during our conversation is that program validation and instruction execution are separate stages. ProgramLoader validates the complete program, InstructionDecoder validates individual instruction encodings, and concrete instruction classes implement their own validation rules. Only a completely valid program can be installed and executed, while runtime validation protects against invalid instructions introduced by later memory modifications.

3.2.3 Final PlantUML Code (ChatGPT)

```
@startuml
title Vole Machine Simulator - Updated Design (Validate Before Install)

skinparam classAttributeIconSize 0
skinparam packageStyle rectangle
hide empty members

enum StepStatus {
```

```
  CONTINUE
  HALTED
  INVALID_INSTRUCTION
  INVALID_PROGRAM_COUNTER
}

package "Application / UI" {
  class SimulatorApp {
    - decoder : InstructionDecoder
    - machine : Machine
    - loader : ProgramLoader
    - ui : ConsoleUI
    + SimulatorApp()
    + run() : void
    - registerInstructionFactories() : void
    - loadProgramFlow() : void
    - editRegisterFlow() : void
    - editMemoryFlow() : void
    - runAllFlow() : void
    - runStepFlow() : void
    - showCurrentState() : void
    - afterRunPrompt() : void
  }

  class ConsoleUI {
    + showMainMenu() : int
    + promptFileName() : string
    + promptStartAddress() : int
    + promptRegisterEdit(outIndex : int, outValue : int) : bool
    + promptMemoryEdit(outAddress : int, outValue : int) : bool
    + showState(machine : Machine) : void
    + showMessage(message : string) : void
    + waitForEnter() : void
    + askLoadAnother() : bool
  }
}

package "Program Loading" {
  class ProgramLoader {
```

```
    + load(fileName : string, startAddress : int, decoder : InstructionDecoder&, outError :
string&) : unique_ptr
    - parseInstructionLine(line : string, outCode : uint16_t, outError : string&) : bool
    - validateEveryInstruction(codes : vector, decoder : InstructionDecoder&, outError :
string&) : bool
    - isHexDigit(ch : char) : bool
    - isEvenAddress(address : int) : bool
    - fitsMemory(startAddress : int, byteCount : size_t) : bool
  }

  class ProgramImage {
    - startAddress : uint8_t
    - bytes : vector
    - ProgramImage(startAddress : uint8_t, bytes : vector)
    + getStartAddress() : uint8_t
    + getBytes() : const vector&
    + size() : size_t
  }
}

package "Machine Core" {
  class Machine {
    - cpu : CPU
    - memory : Memory
    - screen : Screen
    - decoder : InstructionDecoder&
    - lastError : string
    + Machine(decoder : InstructionDecoder&)
    + step() : StepStatus
    + run() : StepStatus
    + installProgram(image : const ProgramImage&) : bool
    + editRegister(index : int, value : int) : bool
    + editMemory(address : int, value : int) : bool
    + getLastError() : string
    + getCPU() : const CPU&
    + getMemory() : const Memory&
    + getScreen() : const Screen&
  }
```

```
class CPU {
  - registers : RegisterBank
  - pc : ProgramCounter
  - ir : InstructionRegister
  - alu : ALU
  - halted : bool
  + readRegister(index : uint8_t) : uint8_t
  + writeRegister(index : uint8_t, value : uint8_t) : void
  + getPC() : uint8_t
  + setPC(address : uint8_t) : void
  + advancePCBy2() : void
  + getIR() : uint16_t
  + setIR(code : uint16_t) : void
  + getALU() : ALU&
  + halt() : void
  + clearHalt() : void
  + isHalted() : bool
}

class RegisterBank {
  - registers : Register8[16]
  + read(index : uint8_t) : uint8_t
  + write(index : uint8_t, value : uint8_t) : void
}

class Register8 {
  - value : uint8_t
  + getValue() : uint8_t
  + setValue(value : uint8_t) : void
}

class ProgramCounter {
  - address : uint8_t
  + getAddress() : uint8_t
  + setAddress(address : uint8_t) : void
  + advanceBy2() : void
}

class InstructionRegister {
```

```
  - code : uint16_t
  + getCode() : uint16_t
  + setCode(code : uint16_t) : void
 }

 class ALU {
  + addTwosComplement(a : uint8_t, b : uint8_t) : uint8_t
  + addFloat8(a : uint8_t, b : uint8_t) : uint8_t
  + bitOr(a : uint8_t, b : uint8_t) : uint8_t
  + bitAnd(a : uint8_t, b : uint8_t) : uint8_t
  + bitXor(a : uint8_t, b : uint8_t) : uint8_t
  + rotateRight(value : uint8_t, count : uint8_t) : uint8_t
 }

 class Memory {
  - cells : uint8_t[256]
  + read(address : uint8_t) : uint8_t
  + writeRaw(address : uint8_t, value : uint8_t) : void
  + loadBytes(startAddress : uint8_t, bytes : vector) : bool
 }

 class Screen {
  - output : vector
  + write(value : uint8_t) : void
  + clear() : void
  + getOutput() : const vector&
 }
}

package "Instruction Decoding and Execution" {
 class InstructionDecoder {
  - factories : map
  + registerFactory(opcode : uint8_t, factory : InstructionFactory) : void
  + decode(code : uint16_t, outError : string&) : unique_ptr
 }

 abstract class Instruction {
  # rawCode : uint16_t
  # d1 : uint8_t
```

```
  # d2 : uint8_t
  # d3 : uint8_t
  # Instruction(rawCode : uint16_t)
  # addressXY() : uint8_t
  + {abstract} isEncodingValid(outError : string&) : bool
  + {abstract} execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class LoadMemoryInstruction {
  + LoadMemoryInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class LoadImmediateInstruction {
  + LoadImmediateInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class StoreInstruction {
  + StoreInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class MoveInstruction {
  + MoveInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class AddIntegerInstruction {
  + AddIntegerInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class AddFloatInstruction {
```

```
  + AddFloatInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class OrInstruction {
  + OrInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class AndInstruction {
  + AndInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class XorInstruction {
  + XorInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class RotateRightInstruction {
  + RotateRightInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class JumpInstruction {
  + JumpInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
}

class HaltInstruction {
  + HaltInstruction(code : uint16_t)
  + isEncodingValid(outError : string&) : bool
  + execute(cpu : CPU, memory : Memory, screen : Screen) : void
```

```
 }
}

SimulatorApp "1" *-- "1" Machine
SimulatorApp "1" *-- "1" InstructionDecoder
SimulatorApp "1" *-- "1" ProgramLoader
SimulatorApp "1" *-- "1" ConsoleUI

Machine "1" *-- "1" CPU
Machine "1" *-- "1" Memory
Machine "1" *-- "1" Screen

Machine --> "1" InstructionDecoder : uses for runtime decoding

CPU "1" *-- "1" RegisterBank
RegisterBank "1" *-- "16" Register8
CPU "1" *-- "1" ProgramCounter
CPU "1" *-- "1" InstructionRegister
CPU "1" *-- "1" ALU

ProgramLoader ..> InstructionDecoder : pre-validates every file instruction
ProgramLoader ..> ProgramImage : creates only after all checks pass
Machine ..> ProgramImage : installs validated image
ConsoleUI ..> Machine : displays state
SimulatorApp ..> ProgramImage : install only after successful load

Instruction <|-- LoadMemoryInstruction
Instruction <|-- LoadImmediateInstruction
Instruction <|-- StoreInstruction
Instruction <|-- MoveInstruction
Instruction <|-- AddIntegerInstruction
Instruction <|-- AddFloatInstruction
Instruction <|-- OrInstruction
Instruction <|-- AndInstruction
Instruction <|-- XorInstruction
Instruction <|-- RotateRightInstruction
Instruction <|-- JumpInstruction
Instruction <|-- HaltInstruction
```

```
Instruction ..> CPU : executes against
Instruction ..> Memory : reads/writes
Instruction ..> Screen : output device

AddIntegerInstruction ..> ALU : arithmetic via CPU
AddFloatInstruction ..> ALU : arithmetic via CPU
OrInstruction ..> ALU : operation via CPU
AndInstruction ..> ALU : operation via CPU
XorInstruction ..> ALU : operation via CPU
RotateRightInstruction ..> ALU : operation via CPU

InstructionDecoder ..> Instruction : creates polymorphically
Machine ..> StepStatus : returns

note right of ProgramLoader
  1. Parse the complete file and check the start address
     and memory bounds.
  2. Decode/validate every instruction before producing
     a ProgramImage. Stop and report the first invalid one.
  3. Return nullptr on any error; memory is untouched.

  ProgramImage is created only if every instruction is valid.
end note

note right of InstructionDecoder
  decode() uses the opcode factory to create a candidate,
  then calls its virtual isEncodingValid().

  Return nullptr for unknown opcodes or invalid encodings.

  The loader discards these temporary objects after checking;
  the machine uses decoded objects for execution.
end note

note right of SimulatorApp
  loadProgramFlow(): call loader.load() first.

  If it returns nullptr, report the error and do not install
  or execute anything. Otherwise call machine.installProgram().
```

```
end note

note right of Machine
  step(): check PC, fetch both bytes into IR, advance PC by 2,
  decode/validate the fetched instruction, then execute it.

  Runtime validation remains necessary because the user can
  edit memory after loading.

  run() stops on any non-CONTINUE status.
end note

note right of Memory
  loadBytes() checks the complete address range before writing,
  so a failed installation cannot partially copy the program.
end note

note right of ProgramImage
  Private constructor; ProgramLoader is its friend in C++.

  Only the loader can create an image after successful validation.
end note

note bottom of StoreInstruction
  For opcode 3, address FF sends the byte to Screen;
  other addresses write Memory.
end note

note bottom of InstructionDecoder
  InstructionFactory is a callable alias that creates one
  Instruction from a 16-bit code.

  Register opcode-to-factory pairs during setup;
  decode() needs no growing opcode switch.
end note

@enduml
```

3.2.4 UML Class Diagram (ChatGPT)

https://github.com/AlaaAl-DeenNabilSarhanAl-Maliki/oop_assignment2_Vole_Machine/blob/main/Chat%20GPT%20design.svg

3.3 AI #3: Claude 3.5 Sonnet

3.3.1 Tool Information

- **Tool:** Claude (Anthropic)
- **Model/version:** Claude 3.5 Sonnet
- **Date of use:** [FILL IN]
- **Input given:** The unified prompt (Vole machine description, the 11 requirements, program file format) + our initial PlantUML design (Section 2.5).
- **Not given:** The original assignment file.
- **Number of rounds:** 5 (Round 0 to Round 4)

3.3.2 Conversation Summary

The full conversation is available at the following link:
https://claude.ai/share/a24eaee0-4be4-413c-b416-54a9c1218590

Round 0: Project Introduction and Initial Design
The conversation began by defining the Vole Machine Simulator project, its requirements, and the team's initial UML design.
Round 1: Reviewing the Initial Design
The initial design was critically evaluated to identify weaknesses in class responsibilities, inheritance, dependencies, and instruction handling.
Round 2: Proposing an Improved UML Design
A redesigned class diagram was developed, introducing clearer responsibilities, improved relationships, and a more organized instruction hierarchy.
Round 3: Explaining Design Decisions
The main design choices were justified, including merging CPU and CU, separating the user interface from the machine, handling memory-mapped I/O inside Memory, and improving instruction validation and execution.
Round 4: Understanding the ALU Design
The discussion concluded by explaining why the ALU was designed as a utility class rather than an independent object, comparing both approaches in terms of state, polymorphism, flexibility, and testing.

3.3.3 Final PlantUML Code (Claude)

```
@startuml VoleMachineSimulator
skinparam classAttributeIconSize 0
skinparam shadowing false
hide empty members

' ==================== UI LAYER ====================
class ConsoleUI {
  -machine : Machine&
  -view : ConsoleView
  +ConsoleUI(machine : Machine&)
  +run() : void
  -showMenu() : void
  -handleLoadProgram() : void
  -handleSetProgramCounter() : void
  -handleEditRegister() : void
  -handleEditMemory() : void
  -handleDisplayState() : void
  -handleExecute() : void
  -runAtOnce() : void
  -runStepByStep() : void
  -askExitOrLoadAnother() : bool
  -readHexValue(prompt : string) : int
}

class ConsoleView {
  +showState(machine : const Machine&) : void
  +showMemory(memory : const Memory&) : void
  +showRegisters(cpu : const Cpu&) : void
  +showScreen(screen : const Screen&) : void
  +showStatus(status : StepStatus) : void
  +showLoadResult(result : const LoadResult&) : void
  -toHex(value : int, digits : int) : string
}

' ==================== FACADE ====================
class Machine {
```

```
  -screen : Screen
  -memory : Memory
  -decoder : InstructionDecoder
  -cpu : Cpu
  -loader : ProgramLoader
  +Machine()
  +loadProgram(path : string, startAddress : uint8_t) : LoadResult
  +setProgramCounter(address : uint8_t) : void
  +writeRegister(index : uint8_t, value : uint8_t) : void
  +writeMemory(address : uint8_t, value : uint8_t) : void
  +step() : StepStatus
  +runAll(maxSteps : int = 100000) : StepStatus
  +reset() : void
  +getMemory() : const Memory&
  +getCpu() : const Cpu&
  +getScreen() : const Screen&
}

' ====================== HARDWARE ======================
class Cpu {
  -registers : RegisterFile
  -programCounter : uint8_t
  -instructionRegister : uint16_t
  -halted : bool
  -memory : Memory&
  -decoder : const InstructionDecoder&
  +Cpu(memory : Memory&, decoder : const InstructionDecoder&)
  +step() : StepStatus
  +reset() : void
  +halt() : void
  +isHalted() : bool
  +getRegisters() : RegisterFile&
  +getRegisters() : const RegisterFile&
  +getMemory() : Memory&
  +getProgramCounter() : uint8_t
  +setProgramCounter(address : uint8_t) : void
  +getInstructionRegister() : uint16_t
  -fetch() : uint16_t
}
```

```
class RegisterFile {
  +{static} COUNT : int = 16
  -values : uint8_t[16]
  +get(index : uint8_t) : uint8_t
  +set(index : uint8_t, value : uint8_t) : void
  +clear() : void
}

class Memory {
  +{static} SIZE : int = 256
  +{static} SCREEN_ADDRESS : uint8_t = 0xFF
  -cells : uint8_t[256]
  -screen : Screen&
  +Memory(screen : Screen&)
  +read(address : uint8_t) : uint8_t
  +write(address : uint8_t, value : uint8_t) : void
  +clear() : void
}

class Screen {
  -output : vector
  +write(value : uint8_t) : void
  +getOutput() : const vector&
  +clear() : void
}

' ====================== INSTRUCTIONS ======================
abstract class Instruction {
  -word : uint16_t
  #Instruction(word : uint16_t)
  #regR() : uint8_t
  #regS() : uint8_t
  #regT() : uint8_t
  #operandXY() : uint8_t
  +getWord() : uint16_t
  +{abstract} execute(cpu : Cpu&) : void
  +~Instruction()
}
```

```
class LoadFromMemoryInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}
class LoadImmediateInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}
class StoreInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}
class MoveInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}

abstract class BinaryRegisterInstruction {
 +execute(cpu : Cpu&) : void
 #{abstract} compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AddIntInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 #compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AddFloatInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 #compute(a : uint8_t, b : uint8_t) : uint8_t
}
class OrInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 #compute(a : uint8_t, b : uint8_t) : uint8_t
}
class AndInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 #compute(a : uint8_t, b : uint8_t) : uint8_t
}
class XorInstruction {
```

```
 +{static} create(word : uint16_t) : unique_ptr
 #compute(a : uint8_t, b : uint8_t) : uint8_t
}

class RotateInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}
class JumpInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}
class HaltInstruction {
 +{static} create(word : uint16_t) : unique_ptr
 +execute(cpu : Cpu&) : void
}

class Alu <> {
 +{static} addInt(a : uint8_t, b : uint8_t) : uint8_t
 +{static} addFloat(a : uint8_t, b : uint8_t) : uint8_t
 +{static} bitOr(a : uint8_t, b : uint8_t) : uint8_t
 +{static} bitAnd(a : uint8_t, b : uint8_t) : uint8_t
 +{static} bitXor(a : uint8_t, b : uint8_t) : uint8_t
 +{static} rotateRight(value : uint8_t, times : uint8_t) : uint8_t
 -{static} floatToDouble(bits : uint8_t) : double
 -{static} doubleToFloat(value : double) : uint8_t
}

class InstructionDecoder {
 -creators : map
 +InstructionDecoder()
 +registerInstruction(opcode : uint8_t, creator : InstructionCreator) : void
 +decode(word : uint16_t) : unique_ptr
 +isValid(word : uint16_t) : bool
}

' ==================== LOADING ====================
class ProgramLoader {
 -decoder : const InstructionDecoder&
```

```
  +ProgramLoader(decoder : const InstructionDecoder&)
  +load(path : string, startAddress : uint8_t, memory : Memory&) : LoadResult
  -readWords(path : string, words : vector&, result : LoadResult&) : bool
  -parseHexWord(text : string, word : uint16_t&) : bool
}

class LoadResult <> {
  +success : bool
  +lineNumber : int
  +message : string
  +wordsLoaded : int
}

enum StepStatus {
  OK
  HALTED
  INVALID_INSTRUCTION
  STEP_LIMIT_REACHED
}

note right of Memory
  write(0xFF, v) forwards v to the Screen
  and does not store it in cells.
  read(0xFF) returns 0 (assumption).
end note

note bottom of InstructionDecoder
  InstructionCreator = function(word) returning
  unique_ptr, or nullptr when the
  unused digits are wrong. Each concrete class
  provides one (static create) and the decoder
  registers it for its opcode.
end note

' ==================== RELATIONSHIPS ====================
ConsoleUI "1" *-- "1" ConsoleView
ConsoleUI "1" --> "1" Machine : drives
ConsoleView ..> Machine : reads
ConsoleView ..> Memory
```

```
ConsoleView ..> Cpu
ConsoleView ..> Screen
ConsoleView ..> StepStatus
ConsoleView ..> LoadResult

Machine "1" *-- "1" Screen
Machine "1" *-- "1" Memory
Machine "1" *-- "1" InstructionDecoder
Machine "1" *-- "1" Cpu
Machine "1" *-- "1" ProgramLoader
Machine ..> LoadResult
Machine ..> StepStatus

Cpu "1" *-- "1" RegisterFile
Cpu "1" --> "1" Memory
Cpu "1" --> "1" InstructionDecoder
Cpu ..> Instruction : creates / executes
Cpu ..> StepStatus
Memory "1" --> "1" Screen

ProgramLoader "1" --> "1" InstructionDecoder
ProgramLoader ..> Memory : writes
ProgramLoader ..> LoadResult

InstructionDecoder ..> Instruction : creates
Instruction ..> Cpu : execute(cpu)

Instruction <|-- LoadFromMemoryInstruction
Instruction <|-- LoadImmediateInstruction
Instruction <|-- StoreInstruction
Instruction <|-- MoveInstruction
Instruction <|-- BinaryRegisterInstruction
Instruction <|-- RotateInstruction
Instruction <|-- JumpInstruction
Instruction <|-- HaltInstruction
BinaryRegisterInstruction <|-- AddIntInstruction
BinaryRegisterInstruction <|-- AddFloatInstruction
BinaryRegisterInstruction <|-- OrInstruction
BinaryRegisterInstruction <|-- AndInstruction
```

```
BinaryRegisterInstruction <|-- XorInstruction

AddIntInstruction ..> Alu
AddFloatInstruction ..> Alu
OrInstruction ..> Alu
AndInstruction ..> Alu
XorInstruction ..> Alu
RotateInstruction ..> Alu
@enduml
```

3.3.4 UML Class Diagram (Claude)

https://github.com/AlaaAl-DeenNabilSarhanAl-Maliki/oop_assignment2_Vole_Machine/blob/main/Claude%20Design.svg

3.4 Comparison of the Three Designs

Naming used in this section:
- Initial = our team's initial design (Initial Design v1, Section 2)
- Gemini = AI #1 final design (Section 3.2)
- ChatGPT = AI #2 final design (Section 3.3)
- AI #3 = AI #3 final design (Section 3.4) [Claude 3.5 Sonnet]

The counts below include the enum, the abstract classes and the 12 concrete instruction classes.

3.4.1 Overview of the Four Designs

Initial (26 elements)
Machine owns CPU, Memory, Screen, ProgramLoader, InstructionDecoder and Menu. The CPU owns 16 Registers, PC, IR, ALU and a CU (control unit). Screen both stores the output and displays the machine state. Each Instruction has isValid() and execute(cpu).

Gemini (27 elements)
Same skeleton as the initial design. Menu was moved out of Machine, an InstructionValidator (static) was added, runAll() was moved from the ControlUnit to the Menu, and Memory forwards writes at address FF to the Screen (memory-mapped I/O). The ControlUnit stays inside the

CPU.

ChatGPT (28 elements)

The CU was removed and Machine runs the cycle (step/run). A SimulatorApp and a ConsoleUI handle the user interface. ProgramLoader validates the whole file and produces a ProgramImage only if every instruction is valid. Registers are modelled with RegisterBank / Register8 and fixed-width types. The decoder uses a registry of factories.

AI #3 (26 elements)

The CU was removed and the Cpu runs the cycle itself. ConsoleUI (input) and ConsoleView (output) are separated. Memory forwards writes at FF to the Screen. The decoder uses a registry of creators; the ALU is a static utility. A BinaryRegisterInstruction base class shares the code of the five register-to-register instructions. LoadResult reports load errors with a line number, and runAll() has a step limit.

3.4.2 Similarities

All four designs:

- Use one abstract Instruction class with 12 concrete subclasses and call execute() polymorphically (Command pattern), so a new instruction does not require changing the existing ones.
- Use a Machine class as a facade that owns the hardware (CPU, Memory, Screen) and the loader / decoder.
- Keep a separate ProgramLoader and a separate InstructionDecoder.
- Use a StepStatus enum to report the result of one machine cycle.
- Keep the ALU operations (add int, add float, or, and, xor, rotate) in a dedicated ALU, not inside the instruction classes.
- Use composition for hardware parts and association/dependency for collaboration.

All three AI designs, in addition:

- Moved the user interface out of the Machine (Menu / ConsoleUI is no longer a part of the machine).
- Stopped using inheritance between Register, ProgramCounter and InstructionRegister.

Two of the three AI designs (ChatGPT and AI #3) removed the CU and with it the circular relation between the CPU and the CU that existed in the initial design. Gemini kept the ControlUnit class, so the circular reference (CPU owns ControlUnit, ControlUnit holds CPU&) remains.

3.4.3 Differences

a) Who runs the machine cycle (fetch - decode - execute)?

Initial: CU inside the CPU (step, runAll, fetch).

Gemini : ControlUnit inside the CPU (step, fetch); runAll moved to Menu.

ChatGPT: Machine.step() / Machine.run().
AI #3 : Cpu.step() / Cpu.fetch(); Machine.step() / runAll() delegate.

b) How is instruction validity checked?

Initial: virtual isValid() inside every Instruction. The object must exist before it can be validated.
Gemini : static InstructionValidator.isValidInstruction(code) checks the raw code before any object is created.
ChatGPT: virtual isEncodingValid() on each Instruction, called by the decoder on a candidate object. ProgramLoader validates every instruction of the file before a ProgramImage is created (all-or-nothing); Machine validates again at run time.
AI #3 : each concrete class has a static create(word) that returns nullptr if the unused digits are wrong; the decoder offers decode() and isValid(); ProgramLoader uses the decoder.

c) How is the screen (address FF) handled?

Initial: CPU --> Screen; the StoreInstruction sends the value to Screen.
Gemini : Memory holds a Screen* and forwards writes at FF (MMIO).
ChatGPT: StoreInstruction decides; execute() receives Screen as a parameter. Memory does not know the Screen.
AI #3 : Memory holds a Screen& and forwards writes at FF (MMIO); read(FF) returns 0 (documented assumption).

d) Signature of execute()

Initial: execute(cpu)
Gemini : execute(cpu&, memory&)
ChatGPT: execute(cpu, memory, screen) (written without & in the diagram)
AI #3 : execute(cpu&); memory is reached through the Cpu.

e) How are registers and special registers represented?

Initial: Register base class; PC and IR inherit from it (8-bit vs 16-bit conflict).
Gemini : three separate classes, values stored as int.
ChatGPT: RegisterBank (16 x Register8), ProgramCounter, InstructionRegister, with uint8_t / uint16_t.
AI #3 : RegisterFile (uint8_t[16]); PC and IR are plain attributes of Cpu (uint8_t and uint16_t).

f) How is a new instruction added (extensibility)?

Initial: decoder maps opcodes to classes (mechanism not specified).
Gemini : decoder creates the objects (mechanism not specified).
ChatGPT: opcode -> factory map; registerFactory(opcode, factory).
AI #3 : opcode -> creator map; registerInstruction(opcode, creator).

g) Duplicate code between similar instructions

Initial, Gemini, ChatGPT: each of the five RST instructions is a separate subclass of Instruction with its own execute().
AI #3: BinaryRegisterInstruction implements execute() once and subclasses only provide compute() (Template Method).

h) User interface
Initial: Menu (part of Machine) plus Screen that also displays state.
Gemini : Menu (independent, all UI in one class).
ChatGPT: SimulatorApp (flows) + ConsoleUI (prompts and display).
AI #3 : ConsoleUI (flows and input) + ConsoleView (display only).
i) Program loading and error reporting
Initial: load() returns bool; lastError string.
Gemini : load() returns bool; lastError string.
ChatGPT: load() returns unique_ptr<ProgramImage> or nullptr plus an error string.
AI #3 : load() returns LoadResult (success, lineNumber, message, wordsLoaded).
j) Protection against endless programs
Initial, Gemini, ChatGPT: none (a program such as "B000" jumping to itself would never stop in run-all mode).
AI #3: runAll(maxSteps = 100000) and the status STEP_LIMIT_REACHED.
k) Data types
Initial and Gemini use int for cells, registers and addresses; ChatGPT and AI #3 use uint8_t / uint16_t, matching the 8-bit and 16-bit sizes of the Vole machine.

3.4.4 Comparison Table

Criterion	Initial	Gemini	ChatGPT	Claude
Number of elements	26	27	28	26
Machine cycle owner	CU (in CPU)	ControlUnit	Machine	Cpu
CPU <-> CU circular reference	Yes	Yes	No (CU removed)	No (CU removed)
UI separated from Machine	No (Menu is a part)	Yes	Yes	Yes
Display separated from input	No	No	Partly	Yes (ConsoleView)
Validation before object creation	No	Yes (static validator)	Partly (decoder creates candidate)	Partly (create() returns nullptr)

Criterion	Initial	Gemini	ChatGPT	Claude
Whole program checked before loading	Not specified	Not specified	Yes (ProgramImage)	Not specified
Run-time re-validation	Not shown	Not shown	Yes (Machine)	Yes (decoder in Cpu)
Memory-mapped screen (FF)	In CPU/Store	In Memory	In StoreInstruction	In Memory
Instructions know Screen	No (CPU only)	No	Yes	No
execute() parameters	cpu	cpu, memory	cpu, memory, screen	cpu
Opcode registry (extensible)	No	No	Yes (factories)	Yes (creators)
Shared base for RST ops	No	No	No	Yes
Register class hierarchy	Inheritance	Separate classes	Separate classes	RegisterFile only
8/16-bit types	int	int	uint8_t / uint16_t	uint8_t / uint16_t
Detailed load result	bool + string	bool + string	nullptr + string	LoadResult struct
Step limit for run-all	No	No	No	Yes
Invalid PC detection	No	No	Yes	No
Explicit "set PC" operation	No	No	CPU.setPC	Machine.setProgramCounter + UI handler
Explicit "wait for Enter"	No	No	Yes (ConsoleUI)	No (inside runStepByStep)

Coverage of the 11 requirements (class and method responsible)

Req	Initial	Gemini	ChatGPT	Claude
1	Menu.showMainMenu	Menu.showMainMenu	ConsoleUI.showMainMenu	ConsoleUI.showMenu
2	ProgramLoader.load, isEvenAddress	ProgramLoader.load, Validator.isEven	ProgramLoader.load, isEvenAddress, fitsMemory	ProgramLoader.load (even-address check not shown as method)
3	Menu.editRegister / editMemory	Menu.handleEdit*, CPU.setRegister	Machine.editRegister, editMemory	Machine.writeRegister, writeMemory
4	implied (PC.setValue)	implied (PC.setValue)	CPU.setPC / installProgram	Machine.setProgramCounter
5	Screen.show*	Menu.displayState	ConsoleUI.showState	ConsoleView.showState, showMemory, ...
6	CU.step, fetch	ControlUnit.step, fetch	Machine.step	Cpu.step, fetch
7	Menu.runAll / runStep	Menu.handleRunAll / handleRunStep	SimulatorApp.runAllFlow / runStepFlow	ConsoleUI.handleExecute, runAtOnce, runStepByStep
8-9	Menu + Screen	Menu	ConsoleUI.showState, waitForEnter	ConsoleView + ConsoleUI
10	Menu.askExitOrLoad	Menu.promptExitOrReload	ConsoleUI.askLoadAnother	ConsoleUI.askExitOrLoadAnother
11	Instruction.isValid	Validator + ProgramLoader	ProgramLoader (all instructions) + run-time check	Decoder.isValid / create() + ProgramLoader

3.4.5 Comparison of Each AI Design with Our Initial Design

Gemini vs Initial

- Kept: the whole structure (Machine, CPU with CU/ALU/registers, Memory, Screen, ProgramLoader, InstructionDecoder, 12 instruction classes).
- Changed: Menu moved out of Machine; Screen no longer displays state; validation moved out of the instruction classes into a static InstructionValidator; the Screen reaches memory writes through Memory (MMIO); execute(cpu, memory); registers no longer inherit from each other.
- Removed: runAll() from the control unit; isValid() from Instruction.
- Added: InstructionValidator.
- Not solved: the CPU <-> ControlUnit circular reference; the dependency of Instruction on ALU was dropped from the diagram.
- Overall: the closest to our initial design and an incremental improvement of it.

ChatGPT vs Initial

- Kept: abstract Instruction with 12 subclasses, Machine, Memory, Screen, ALU, ProgramLoader, InstructionDecoder.
- Changed: validation became a joint job of ProgramLoader (whole program), InstructionDecoder and isEncodingValid(); the decoder uses a factory registry; fixed-width types; registers split into RegisterBank/Register8.
- Removed: the CU class and the Menu inside Machine.
- Added: SimulatorApp, ConsoleUI, ProgramImage, RegisterBank, Register8, INVALID_PROGRAM_COUNTER status, validate-before-install workflow.
- Overall: the most thorough on validation and data types, but it has the most classes and a larger change from the initial design.

Claude vs Initial

- Kept: abstract Instruction with 12 subclasses, Machine, Memory, Screen, ProgramLoader, InstructionDecoder.
- Changed: the CPU runs the cycle itself; the ALU became a static utility; Memory forwards writes at FF to the Screen; the decoder uses a creator registry; registers no longer use inheritance; UI and display are two classes.
- Removed: CU, Menu inside Machine, Register / ProgramCounter / InstructionRegister classes, display methods inside Screen.
- Added: ConsoleView, RegisterFile, BinaryRegisterInstruction, LoadResult, STEP_LIMIT_REACHED, explicit set-PC operation.
- Overall: a different and simpler structure than the initial design with the same number of elements.

3.5.6 Conclusion of the Comparison

All three AI designs improved the initial design in the same two main points: the UI is no longer part of the machine and the register inheritance problem is gone. They differ mainly in where the machine cycle lives, how validation is organised and how the screen at address FF is connected. Claude gives the best balance of requirement coverage, coupling and simplicity; ChatGPT gives the strongest validation workflow; Gemini stays closest to our initial design but leaves several problems open.

4. FINAL DESIGN

4.1 Selection

1 = weak, 5 = strong

Criterion	Initial	Gemini	ChatGPT	Claude
Coverage of the 11 requirements	3	3	4	5
Low coupling / encapsulation	2	3	3	4
Extensibility (new instruction)	3	4	5	5
Simplicity (no over-engineering)	4	4	2	4
Fidelity to hardware (8/16 bits)	2	2	5	4
Validation robustness	2	3	5	4
UI separation	2	4	4	5

Criterion	Initial	Gemini	ChatGPT	Claude
Total (max 35)	**18**	**23**	**28**	**31**

Choice: Claude as the base design.
Reasons:

1. Best coverage of the requirements: explicit set-PC operation, load result with line number, run-all step limit.
2. Lowest coupling: instructions only receive the Cpu, the screen is hidden behind Memory, display is separated from input.
3. Less duplicated code: BinaryRegisterInstruction.
4. Extensible: opcode registry in the decoder.
5. Same size as our initial design (26 elements), so it is not over-engineered.

4.2 Enhancement

Chat link: *[With Claude]*
https://claude.ai/share/25dce69c-67da-4679-9cf6-b58720b6fe0d

4.3 Final PlantUML code

```
@startuml
skinparam classAttributeIconSize 0
skinparam linetype ortho

' ==================== Core ====================

enum StepStatus {
  OK
  HALTED
```

```
 INVALID_INSTRUCTION
 INVALID_PROGRAM_COUNTER
 STEP_LIMIT_REACHED
}

class LoadResult <<struct>> {
 + success : bool
 + lineNumber : int
 + message : string
 + wordsLoaded : int
}

class Machine {
 - screen : Screen
 - memory : Memory
 - decoder : InstructionDecoder
 - cpu : Cpu
 - loader : ProgramLoader
 + Machine()
 + loadProgram(path : string, startAddress : int) : LoadResult
 + setProgramCounter(address : uint8_t) : bool
 + writeRegister(index : uint8_t, value : uint8_t) : bool
 + writeMemory(address : uint8_t, value : uint8_t) : void
 + step() : StepStatus
 + runAll(maxSteps : int = 100000) : StepStatus
 + reset() : void
 + getMemory() : const Memory&
```

```
 + getScreen() : const Screen&
 + getCpu() : const Cpu&
}

class Cpu {
 - registers : RegisterFile
 - programCounter : uint8_t
 - instructionRegister : uint16_t
 - halted : bool
 - memory : Memory&
 - decoder : InstructionDecoder&
 - fetch() : uint16_t
 + Cpu(memory : Memory&, decoder : InstructionDecoder&)
 + step() : StepStatus
 + resume() : void
 + resetState() : void
 + halt() : void
 + isHalted() : bool
 + readRegister(index : uint8_t) : uint8_t
 + writeRegister(index : uint8_t, value : uint8_t) : void
 + readMemory(address : uint8_t) : uint8_t
 + writeMemory(address : uint8_t, value : uint8_t) : void
 + getRegisters() : const RegisterFile&
 + getProgramCounter() : uint8_t
 + setProgramCounter(address : uint8_t) : void
 + getInstructionRegister() : uint16_t
}
```

```
class RegisterFile {
 + {static} count : int = 16
 - Registers : uint8_t[16]
 + get(index : uint8_t) : uint8_t
 + set(index : uint8_t, value : uint8_t) : void
 + clear() : void
}

class Memory {
 + {static} Size : int = 256
 + {static} screenAddress : uint8_t = 0xFF
 - cells : uint8_t[256]
 - screen : Screen&
 + Memory(screen : Screen&)
 + {static} isEvenAddress(address : int) : bool
 + {static} fits(start : int, byteCount : int) : bool
 + write(address : uint8_t, value : uint8_t) : void
 + read(address : uint8_t) : uint8_t
 + loadBytes(start : int, bytes : vector<uint8_t>) : bool
 + clear() : void
}

class Screen {
 - Output : vector<uint8_t>
 + write(value : uint8_t) : void
 + getOutput() : const vector<uint8_t>&
```

```
 + clear() : void
}

class ProgramLoader {
 - decoder : const InstructionDecoder&
 - readWords(path, words, lineNumbers, result) : bool
 - ParseHexWord(text : string, word : uint16_t&) : bool
 - toHexText(value : int, digitCount : int) : string
 - validateAll(words, lineNumbers, startAddress, result) : bool
 + ProgramLoader(decoder : const InstructionDecoder&)
 + load(path : string, startAddress : int, memory : Memory&) : LoadResult
}

class InstructionDecoder {
 - creators : map<uint8_t, InstructionCreator>
 + InstructionDecoder()
 + registerInstruction(opcode : uint8_t, creator : InstructionCreator) : void
 + decode(word : uint16_t) : unique_ptr<Instruction>
 + isValid(word : uint16_t) : bool
}

class Alu {
 + {static} addInt(a, b : uint8_t) : uint8_t
 + {static} addFloat(a, b : uint8_t) : uint8_t
 + {static} bitOR(a, b : uint8_t) : uint8_t
 + {static} bitAnd(a, b : uint8_t) : uint8_t
 + {static} bitXor(a, b : uint8_t) : uint8_t
```

```
 + {static} rotateRight(value, times : uint8_t) : uint8_t
 + {static} floatToDouble(bits : uint8_t) : double
 + {static} doubleToFloat(value : double) : uint8_t
}

' ===================== Instructions =====================

abstract class Instruction {
 - word : uint16_t
 # Instruction(word : uint16_t)
 # regR() : uint8_t
 # regS() : uint8_t
 # regT() : uint8_t
 # operandXY() : uint8_t
 + getWord() : uint16_t
 + {abstract} execute(cpu : Cpu&) : void
 + ~Instruction()
}

class LoadFromMemoryInstruction {
 + {static} create(word : uint16_t) : unique_ptr<Instruction>
 + execute(cpu : Cpu&) : void
}
class LoadImmediateInstruction {
 + {static} create(word : uint16_t) : unique_ptr<Instruction>
 + execute(cpu : Cpu&) : void
}
```

```
class StoreInstruction {
  + {static} create(word : uint16_t) : unique_ptr<Instruction>
  + execute(cpu : Cpu&) : void
}
class MoveInstruction {
  + {static} create(word : uint16_t) : unique_ptr<Instruction>
  + execute(cpu : Cpu&) : void
}
class RotationInstruction {
  + {static} create(word : uint16_t) : unique_ptr<Instruction>
  + execute(cpu : Cpu&) : void
}
class JumpInstruction {
  + {static} create(word : uint16_t) : unique_ptr<Instruction>
  + execute(cpu : Cpu&) : void
}
class HaltInstruction {
  + {static} create(word : uint16_t) : unique_ptr<Instruction>
  + execute(cpu : Cpu&) : void
}

abstract class BinaryRegisterInstruction {
  # BinaryRegisterInstruction(word : uint16_t)
  + execute(cpu : Cpu&) : void
  # {abstract} compute(a, b : uint8_t) : uint8_t
}
class AndInstruction {
```

```
 + {static} create(word : uint16_t) : unique_ptr<BinaryRegisterInstruction>
 + compute(a, b : uint8_t) : uint8_t
}
class OrInstruction {
 + {static} create(word : uint16_t) : unique_ptr<BinaryRegisterInstruction>
 + compute(a, b : uint8_t) : uint8_t
}
class XorInstruction {
 + {static} create(word : uint16_t) : unique_ptr<BinaryRegisterInstruction>
 + compute(a, b : uint8_t) : uint8_t
}
class AddIntInstruction {
 + {static} create(word : uint16_t) : unique_ptr<BinaryRegisterInstruction>
 + compute(a, b : uint8_t) : uint8_t
}
class AddFloatInstruction {
 + {static} create(word : uint16_t) : unique_ptr<BinaryRegisterInstruction>
 + compute(a, b : uint8_t) : uint8_t
}

' ====================== CLI ======================

class ConsoleUI {
 - machine : Machine&
 - view : ConsoleView
 - currentFile : string
 - loadedStart : int
```

```
  - screenShown : size_t
  + ConsoleUI(machine : Machine&)
  + run() : void
  - execute(args : vector<string>) : bool
  - printHelp() : void
  - loadFile(path : string, startAddress : int) : bool
  - cmdLoad(args) : void
  - cmdStep(args) : void
  - cmdRun() : void
  - cmdRestart() : void
  - cmdReset() : void
  - cmdSetProgramCounter(args) : void
  - cmdSetRegister(args) : void
  - cmdSetMemory(args) : void
}

class ConsoleView {
  + showMemory(memory : const Memory&) : void
  + showScreen(screen : const Screen&) : void
  + showRegisters(cpu : const Cpu&) : void
  + showState(machine : const Machine&) : void
  + showStatus(status : StepStatus) : void
  + showLoadResult(result : const LoadResult&) : void
  - {static} toHex(value : int, digits : int) : string
}

' ===================== Relations =====================
```

```
' Composition (Machine owns its parts)
Machine *-- Screen
Machine *-- Memory
Machine *-- InstructionDecoder
Machine *-- Cpu
Machine *-- ProgramLoader

Cpu *-- RegisterFile
Cpu o-- Memory : <<reference>>
Cpu o-- InstructionDecoder : <<reference>>
Memory o-- Screen : <<reference>>
ProgramLoader o-- InstructionDecoder : <<reference>>

ProgramLoader ..> Memory : loads into
ProgramLoader ..> LoadResult
Machine ..> LoadResult
Machine ..> StepStatus
Cpu ..> StepStatus

' Decoder creates instructions
InstructionDecoder ..> Instruction : creates
Instruction ..> Cpu : execute(cpu)

' Inheritance
Instruction <|-- LoadFromMemoryInstruction
Instruction <|-- LoadImmediateInstruction
```

```
Instruction <|-- StoreInstruction

Instruction <|-- MoveInstruction

Instruction <|-- RotationInstruction

Instruction <|-- JumpInstruction

Instruction <|-- HaltInstruction

Instruction <|-- BinaryRegisterInstruction

BinaryRegisterInstruction <|-- AndInstruction

BinaryRegisterInstruction <|-- OrInstruction

BinaryRegisterInstruction <|-- XorInstruction

BinaryRegisterInstruction <|-- AddIntInstruction

BinaryRegisterInstruction <|-- AddFloatInstruction


' Alu usage (assumed from compute() implementations)

AndInstruction ..> Alu

OrInstruction ..> Alu

XorInstruction ..> Alu

AddIntInstruction ..> Alu

AddFloatInstruction ..> Alu

RotationInstruction ..> Alu


' CLI

ConsoleUI o-- Machine : <<reference>>

ConsoleUI *-- ConsoleView

ConsoleView ..> Machine

ConsoleView ..> Memory

ConsoleView ..> Screen

ConsoleView ..> Cpu
```

`@enduml`

4.4 Final UML class diagram

[INSERT IMAGE HERE: diagram rendered from the final PlantUML code above at plantuml.com]

4.5 Description of the final classes and relationships

User interface

ConsoleUI: runs the menu loop, reads and range-checks user input (readHexValue), and calls Machine. It contains no machine logic. waitForEnter() supports step-by-step execution.
ConsoleView: only prints (state, memory, registers, screen, status, load result).

Facade

Machine: the single entry point to the simulator. It owns Screen, Memory, InstructionDecoder, Cpu and ProgramLoader (declared in this order so each exists before the object that references it). User edits are validated here: setProgramCounter and writeRegister return bool.

Hardware

Cpu: has the registers, PC, IR and halted flag. step() checks the PC, fetches, decodes, and executes. It exposes memory and registers only through readX / writeX.
RegisterFile: 16 byte-sized registers.
Memory: 256 cells. Writing to 0xFF is forwarded to Screen (memory-mapped output). It provides the static checks isEvenAddress and fits, and loadBytes for installing a validated program.
Screen: stores the program's output.

Instructions

Instruction (abstract): holds the 16-bit word, provides helpers (regR/S/T, operandXY), and declares execute(cpu).
BinaryRegisterInstruction (abstract): the shared execute for the five R = S op T instructions; subclasses implement only compute.
Concrete instructions: Load (memory/immediate), Store, Move, AddInt, AddFloat, Or, And,

Xor, Rotate, Jump, Halt. Each has a static create that returns nullptr when its unused digits are wrong.
Alu (utility): stateless static operations, including the floating-point conversion.
InstructionDecoder: maps an opcode to a creator function. decode returns unique_ptr<Instruction> or nullptr; isValid is decode != nullptr.

Loading

ProgramLoader: parses the file, validates every word, and installs the program only if everything is valid (all-or-nothing).
LoadResult: success flag, line number, message, and word count.
StepStatus: OK, HALTED, INVALID_INSTRUCTION, INVALID_PROGRAM_COUNTER, STEP_LIMIT_REACHED.

Relationships

- **Composition:** ConsoleUI -> ConsoleView; Machine -> its five parts; Cpu -> RegisterFile.
- **Association:** ConsoleUI -> Machine; Cpu -> Memory and InstructionDecoder; Memory -> Screen; ProgramLoader -> InstructionDecoder.
- **Inheritance:** all instructions extend Instruction; the five binary ones extend BinaryRegisterInstruction.
- **Dependency:** Cpu and InstructionDecoder create Instruction; Instruction uses Cpu; the arithmetic and logic instructions use Alu; ConsoleView reads the model and prints StepStatus and LoadResult.

4.6 What changed

From the initial design (v1)

Initial design	Final design
Machine contained Menu; Screen both displayed and stored output	UI split into ConsoleUI + ConsoleView; Machine is a pure facade; Screen only stores output

Initial design	Final design
CU and runAll inside the CPU, plus Register inheritance for PC and IR	Cpu has step() with PC/IR as plain uint8_t / uint16_t; RegisterFile replaces separate register classes
int everywhere	Exact types (uint8_t, uint16_t) matching the machine
isValid() inside every instruction	Static create() returns nullptr for invalid encodings, so no invalid object exists
decode() with a growing switch (returns a raw pointer)	Registration-based decoder returning unique_ptr
Instruction classes duplicated the ALU calls	BinaryRegisterInstruction (shared execute) + static Alu
Output at FF handled by CPU --> Screen	Memory-mapped I/O inside Memory
Errors as a bool + lastError	LoadResult (line number + message) and richer StepStatus

From the chosen AI design (Claude), per Section 4.2

Change	Reason
All-or-nothing loading (validateAll, Memory::loadBytes)	No partially loaded program on error
Run-time validation in Cpu::step()	Memory can be edited after loading
New INVALID_PROGRAM_COUNTER	Odd PC cannot start a 2-byte instruction
Memory::isEvenAddress / fits; Jump create() rejects odd XY	Explicit, single-place address rules
setProgramCounter and writeRegister return bool	Invalid user edits are rejected
Cpu::getMemory() / non-const getRegisters() replaced by readX / writeX	Encapsulation: no mutable internals exposed
ConsoleUI::waitForEnter(); readHexValue takes a maximum	Clear step-by-step flow, input range checked at the UI

Change	Reason
Notes on ownership (unique_ptr destroyed at end of step()), virtual destructor; <<utility>> / <<struct>> stereotypes	Clear object lifetime; valid PlantUML

Considered and not adopted: ChatGPT's ProgramImage with a private constructor, and separate Register8 / ProgramCounter / InstructionRegister classes. Both add complexity without new behavior. Nothing structural was taken from Gemini (its memory-mapped output idea was already in this design).

4.7 Why this is the best design

- **Separation of concerns:** UI, facade, hardware, instructions, and loading are separate; each class has one job.
- **Open for extension:** adding an instruction means one new class plus one registerInstruction call. No switch needs editing (Open/Closed Principle).
- **Safe by construction:** invalid instructions never become objects, programs load all-or-nothing, and the CPU re-validates at run time.
- **Encapsulation:** the CPU exposes no mutable Memory&; the UI talks only to Machine.
- **Clear ownership:** unique_ptr with temporary lifetime, and a defined construction order in Machine. No leaks and no shared ownership.
- **Faithful to the machine:** exact byte and word types, memory-mapped screen at 0xFF, 2-byte instructions at even addresses.
- **Simple:** it avoids unneeded classes while keeping every required behavior.

4.8 Design Changes During Implementation

While implementing the simulator, we revised the Part 1 design. This section lists each change and the reason for it.

1. Console Interface (`ConsoleUI`)

Initial design	Final implementation
`showMenu()`, `handleLoadProgram()`, `handleSetProgramCounter()`, `handleEditRegister()`, `handleEditMemory()`, `handleDisplayState()`, `handleExecute()`	`loadFile()`, `cmdLoad()`, `cmdRestart()`, `cmdReset()`, `cmdSetProgramCounter()`, `cmdSetRegister()`, `cmdSetMemory()`, `execute(args)`, `printHelp()`

`runAtOnce()`, `runStepByStep()`	`cmdRun()`, `cmdStep(args)`
`askExitOrLoadAnother()`, `readHexValue()`	Removed
(none)	`currentFile`, `loadedStart`, `screenShown`

Rationale: The menu-driven interface was replaced by one-line commands (e.g., `step 3`, `load prog.txt 10`), which make the simulator faster to use. The `restart` command re-runs the last loaded program without retyping its path, which is why `currentFile` and `loadedStart` were added. `screenShown` tracks how many screen bytes were already printed, so they are not repeated during stepping.

2. Machine

Initial design	Final implementation
`loadProgram(path, startAddress : uint8_t)`	`startAddress : int`
`setProgramCounter(...) : void`	`: bool`
`writeRegister(...) : void`	`: bool`

Rationale: `uint8_t` wraps around silently (`0x100` becomes `0x00`). Using `int` allows validating that the start address is within range and **even**, as the assignment requires. Returning `bool` lets the UI report invalid input, such as a register index greater than `F`. `writeMemory()` stays `void`, since every address from `00` to `FF` is valid.

3. Cpu

Initial design	Final implementation	Rationale
`getRegisters()` (non-const), `getMemory()`	Removed	They exposed the CPU's internals.
(none)	`readRegister()`, `writeRegister()`, `readMemory()`, `writeMemory()`	Instructions interact with the CPU through a clear interface (stronger encapsulation).
`reset()`	`resetState()`	Clearer name.
(none)	`resume()`	Continue execution after `HALT`, e.g., after the PC is modified.

`decoder : const InstructionDecoder &`	`InstructionDecoder&`	Changed during implementation.

4. Memory

Initial design	Final implementation
Constants `SIZE`, `SCREEN_ADDRESS`	`static` members `Size`, `screenAddress`
(none)	`isEvenAddress()`, `fits()`, `loadBytes()`

Rationale: Validation that the load address is even and that the program fits in memory now lives next to the data. `loadBytes()` writes the whole program in one call. The trade-off is that `Memory` takes on an extra responsibility, which weakens the Single Responsibility Principle.

5. ProgramLoader

Initial design	Final implementation
`readWords(path, words, result)`	`readWords(path, words, lineNumbers, result)`
(none)	`validateAll()`, `toHexText()`
`parseHexWord`	`ParseHexWord`
`startAddress : uint8_t`	`startAddress : int`

Rationale: Loading is now two-phase: all words are read first, then **all** are validated before anything is written to memory. A faulty program therefore never remains partially loaded. `lineNumbers` enables precise error messages (e.g., “Invalid instruction at line 7”), and `toHexText()` formats hex values in those messages.

6. StepStatus

A new value, `INVALID_PROGRAM_COUNTER`, was added.

Rationale: Each instruction is two bytes long. If `PC = FF`, the fetch would need address `100`, which is outside memory. A `Jump` to an odd address is also problematic. `INVALID_INSTRUCTION` does not cover these cases, because the fault is in the address, not in the instruction.

7. Instruction Classes

Initial design	Final implementation
`create()` of binary instructions returns `unique_ptr<Instruction>`	Returns `unique_ptr<BinaryRegisterInstruction>`

`compute()` is `protected` in subclasses	`public` in subclasses
`RotateInstruction`	`RotationInstruction`
(none)	Protected constructor in `BinaryRegisterInstruction`

Note: The new return type remains compatible with `InstructionCreator`, because `unique_ptr<Derived>` converts implicitly to `unique_ptr<Base>`.

8. Alu

Initial design	Final implementation
`bitOr`	`bitOR`
`floatToDouble`, `doubleToFloat` are `private`	`public static`
Stereotype `«utility»`	Regular class with static methods

Rationale: The two conversion helpers may be useful outside `Alu`, for example for testing or for displaying floating-point values.

9. Naming Changes

Class	Initial design	Final implementation
`RegisterFile`	`COUNT`, `values`	`count`, `Registers`
`Screen`	`output`	`Output`

10. Design Notes (documented assumptions)

- `write(0xFF, v)` forwards the value to the `Screen` and does not store it in `cells`.
- `read(0xFF)` returns `0` (assumption).
- `InstructionCreator` is a function taking a 16-bit word and returning `unique_ptr<Instruction>`, or `nullptr` when the unused digits are wrong (e.g., `C000`, `A R0X`). Each concrete class provides a static `create()`, and the decoder registers it for its opcode.

Summary

The main improvements are stricter address validation (`int` instead of `uint8_t`), two-phase program loading with line-accurate errors, a clearer `Cpu` interface, and the new

`INVALID_PROGRAM_COUNTER` status. The most significant change is the interface style: menu-driven in the initial design, command-driven in the final implementation.

5. EXAMPLES

5.1 Program #1

Purpose:
Swap the values of two registers. Put 05 in R1 and 06 in R2, then swap them using MOVE and R3 as a temporary place.

Code:

```
; ------------------------------------------------------------
; Vole Machine Program: Swap two registers
; Purpose: put 05 in register 1 and 06 in register 2, then swap
; their values using MOVE and register 3 as a temporary place.
; Expected result: R1 = 06, R2 = 05
; Load address: any even address (this program has no jumps)
; ------------------------------------------------------------

2105   ; R1 = 05
2206   ; R2 = 06
2300   ; R3 = 00 (temporary register)
4013   ; MOVE R1 into R3 (save the old R1)
4021   ; MOVE R2 into R1
4032   ; MOVE R3 into R2 (move the saved old R1 to R2)
2300   ; R3 = 00 (clear the temporary register)
C000   ; HALT
```

Expected output:

Nothing is printed on the screen. The final register values are:
R1 = 06
R2 = 05
R3 = 00

5.2 Program #2

Purpose:
Add the numbers 1+2+3+4+5 using a loop, and show the result (15 = 0F in hex) on the screen.

Code:

```
; ------------------------------------------------------------
; Purpose: add the numbers 1+2+3+4+5 using a loop
; and show the result (15 = 0F hex) on the screen.
; Load address: 00 (the jump addresses 08 and 10 are absolute)
; ------------------------------------------------------------

2006   ; R0 = 06 (stop when the counter reaches 6)
2100   ; R1 = 00 (running sum)
2201   ; R2 = 01 (counter)
2301   ; R3 = 01 (the constant one)
B210   ; LOOP START (addr 08): if R2 == R0, jump to the end (addr 10)
5112   ; R1 = R1 + R2   (add counter to sum)
5223   ; R2 = R2 + R3   (counter + 1)
B008   ; R0 == R0 is always true, so always jump back to addr 08
31FF   ; (addr 10) screen <- R1
C000   ; HALT
```

Expected output:
The screen shows 0F (decimal 15). The final register values are:
R0 = 06
R1 = 0F

R2 = 06
R3 = 01

6. EXPERIENCE AND LESSONS LEARNED

What we learned about design
Designing the initial version ourselves showed us how much a design depends on small decisions. For example, who owns the decoder, where the screen output at address FF is handled, and where validation happens. Comparing it with the AI designs taught us to separate responsibilities clearly: UI, a facade (`Machine`), hardware, instructions, and loading. We also learned the value of patterns that keep a design extensible. A decoder that registers a creator per opcode, and a shared base class for the binary instructions, mean a new instruction needs one new class and no `switch` to edit. Finally, we saw that careful design is mostly about edge cases: invalid instructions, an odd program counter, and a program that does not fit in memory.

What we learned about working with AIs
Each AI had a different strength, and no single output was complete. One design had the best validation ideas (all-or-nothing loading, run-time checks), another had a good memory-mapped output idea, and the chosen one had the cleanest structure and ownership model. AI output was most useful when we gave it a specific task, such as a list of enhancements to apply or a clear fix to discuss. It was least useful when the request was open-ended. We also learned that AI suggestions are not automatically correct: some added complexity without adding behavior (extra classes for single-byte registers, a `ProgramImage` with a private constructor). So we had to judge each suggestion ourselves and adopt it only when we could explain the reason.

Problems we faced
Merging ideas from different designs. Each design used different names, types, and

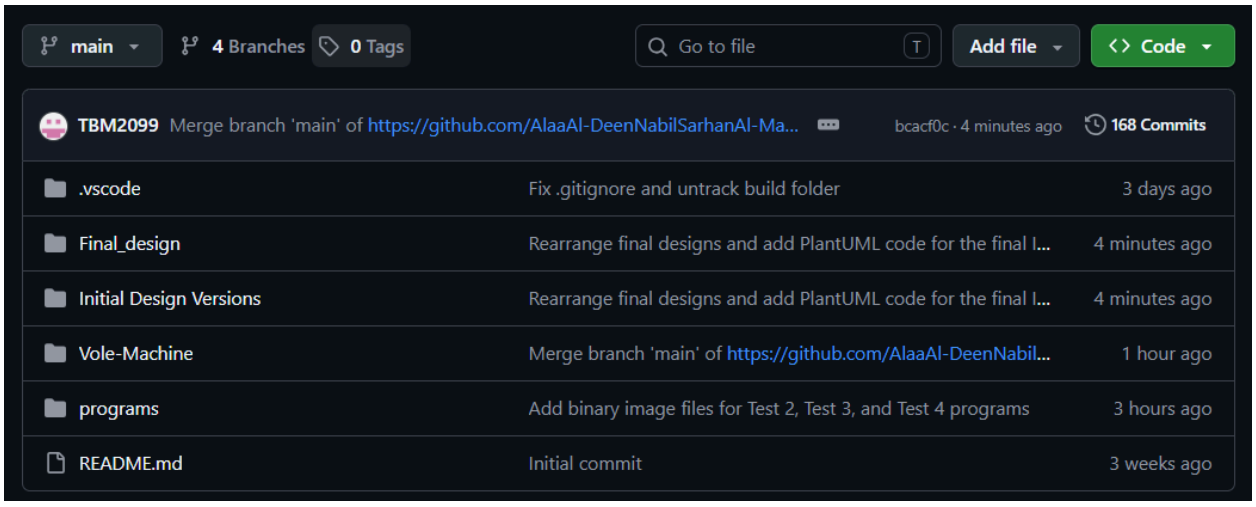
responsibilities, so combining them without breaking consistency took care.
Hidden design flaws. Some problems only appeared when we reviewed the details, such as the CPU exposing a mutable `Memory&`, or a program being partly loaded before an invalid instruction was found.
Ambiguous machine rules. Some behaviors were not stated in the specification (what reading address FF returns, what happens when the PC passes the end of memory), so we had to choose and document an assumption.
Checking the diagram. Keeping the PlantUML code, the class descriptions, and the sample programs consistent with each other required repeated review.

GitHub Project Screenshots

Repository link:
https://github.com/AlaaAl-DeenNabilSarhanAl-Maliki/oop_assignment2_Vole_Machine.git

Repository Structure

Figure 1: Main page of the repository showing the project folders and files.



Project Structure

```
oop_assignment2_Vole_Machine/
├── Final_design/          # PlantUML sources and diagrams (.puml, .svg)
├── Initial Design Versions/ # Part 1 designs
├── README.md
├── Vole-Machine/
│   ├── CMakeLists.txt
│   ├── Core/              # Simulator logic (Include/ + src/)
│   │   └── Alu, Cpu, Memory, Screen, RegisterFile, Machine,
│   │       ProgramLoader, InstructionDecoder, Instruction classes
│   ├── CLI/               # Console interface (ConsoleUI, ConsoleView, main)
│   └── GUI/               # Graphical interface (MainWindow, panels, themes)
└── programs/              # Test_1 ... Test_20 (20 sample Vole programs)
```

Contributors

Figure 3: Contributors page showing the work of each team member.



Successful Execution of Sample Programs

Each program below was loaded into the simulator and executed successfully. For each one we show the program file, the machine state after execution, and the screen output.

Sample Programs: Test Cases

All programs were loaded at address `00` and executed with the final simulator. Together they cover all twelve instruction types (opcodes `1` to `C`), including wrap-around addition, floating-point addition, conditional and unconditional jumps, rotation, and memory access.

Test 1: Swap Two Registers (Using a Temporary Register)

Purpose: Put `05` in register 1 and `06` in register 2, then swap their values using `MOVE` and register 3 as a temporary place.

Program file:

```
None
; Swap two registers

2105 ; R1 = 05

2206 ; R2 = 06

2300 ; R3 = 00 (temporary register)

4013 ; MOVE R1 into R3 (save the old R1)

4021 ; MOVE R2 into R1

4032 ; MOVE R3 into R2 (move the saved old R1 to R2)

2300 ; R3 = 00 (clear the temporary register)

C000 ; HALT
```

Expected result: R1 = `06`, R2 = `05`, R3 = `00`. Nothing is printed to the screen.

Figure 5: Execution of Test 1.

```
================ MACHINE STATE ================
--- CPU Registers ---
PC: 10 | IR: C000

R0: 00  R1: 06  R2: 05  R3: 00
R4: 00  R5: 00  R6: 00  R7: 00
R8: 00  R9: 00  RA: 00  RB: 00
RC: 00  RD: 00  RE: 00  RF: 00
---------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 21 05 22 06 23 00 40 13 40 21 40 32 23 00 C0 00
10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:


ASCII: (Invalid ascii or null will be replaced with '.')

--------------------
================================================
```

Test 2: Sum of 1 to 5 Using a Loop

Purpose: Add the numbers 1+2+3+4+5 using a loop and show the result (15 = `0F`) on the screen.

Program file:

```
None
2006 ; 00: R0 = 06 (stop when the counter reaches 6)

2100 ; 02: R1 = 00 (running sum)

2201 ; 04: R2 = 01 (counter)

2301 ; 06: R3 = 01 (the constant one)

B210 ; 08: LOOP START: if R2 == R0, jump to the end (10)

5112 ; 0A: R1 = R1 + R2 (add counter to sum)

5223 ; 0C: R2 = R2 + R3 (counter + 1)

B008 ; 0E: R0 == R0 is always true, so jump back to 08

31FF ; 10: screen <- R1

C000 ; 12: HALT
```

Expected result: Screen: `0F`. Final registers: R0 = `06`, R1 = `0F`, R2 = `06`, R3 = `01`.

Figure 6: Execution of Test 2.

```
vole> state

================ MACHINE STATE ================
--- CPU Registers ---
PC: 14 | IR: C000

R0: 06  R1: 0F  R2: 06  R3: 01
R4: 00  R5: 00  R6: 00  R7: 00
R8: 00  R9: 00  RA: 00  RB: 00
RC: 00  RD: 00  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 20 06 21 00 22 01 23 01 B2 10 51 12 52 23 B0 08
10: 31 FF C0 00 00 00 00 00 00 00 00 00 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:

ASCII: (Invalid ascii or null will be replaced with '.')
.
--------------------
================================================
```

Test 3: Full Instruction Coverage (Arithmetic, Logic, Rotate, Memory)

Purpose: Test integer add (with wrap-around), OR, AND, XOR, rotate, move, store to memory, load from memory, and screen output. No jumps.

Program file:

None

```
2005 ; R0 = 05

2107 ; R1 = 07
```

```
5201 ; R2 = R0 + R1 = 0C

32FF ; screen <- R2 (prints 0C)

23FD ; R3 = FD (-3 in two's complement)

5403 ; R4 = R0 + R3 = 05 + FD = 02 (carry out is dropped)

34FF ; screen <- R4 (prints 02)

25F0 ; R5 = F0

263C ; R6 = 3C

7756 ; R7 = R5 OR R6 = FC

37FF ; screen <- R7 (prints FC)

8856 ; R8 = R5 AND R6 = 30

38FF ; screen <- R8 (prints 30)

9956 ; R9 = R5 XOR R6 = CC

39FF ; screen <- R9 (prints CC)

2A01 ; RA = 01

AA03 ; rotate RA right 3 times: 01 -> 80 -> 40 -> 20

3AFF ; screen <- RA (prints 20)

A504 ; rotate R5 right 4 times: F0 -> 0F

35FF ; screen <- R5 (prints 0F)

406B ; move R6 into RB (RB = 3C)

3BFF ; screen <- RB (prints 3C)

2C99 ; RC = 99
```

```
3C80 ; memory[80] = RC

1D80 ; RD = memory[80] = 99

3DFF ; screen <- RD (prints 99)

C000 ; halt
```

Expected result: Screen: `0C 02 FC 30 CC 20 0F 3C 99`.

Figure 7: Execution of Test 3.

```
================ MACHINE STATE ================
--- CPU Registers ---
PC: 36 | IR: C000

R0: 05  R1: 07  R2: 0C  R3: FD
R4: 02  R5: 0F  R6: 3C  R7: FC
R8: 30  R9: CC  RA: 20  RB: 3C
RC: 99  RD: 99  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 20 05 21 07 52 01 32 FF 23 FD 54 03 34 FF 25 F0
10: 26 3C 77 56 37 FF 88 56 38 FF 99 56 39 FF 2A 01
20: AA 03 3A FF A5 04 35 FF 40 6B 3B FF 2C 99 3C 80
30: 1D 80 3D FF C0 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 99 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:
0C 02 FC 30 CC 20 0F 3C 99

ASCII: (Invalid ascii or null will be replaced with '.')
...0. .<.
--------------------
================================================
```

Test 4: Loops, Jumps, and Floating-Point Addition

Purpose: Test a loop (1+2+3+4+5), conditional and unconditional jumps, a skipped instruction, and floating-point addition.

Program file:

None

```
2005 ; 00: R0 = 05 (limit, also used by JUMP comparisons)
```

```
2100 ; 02: R1 = 00 (counter)

2200 ; 04: R2 = 00 (sum)

2301 ; 06: R3 = 01 (constant one)

5113 ; 08: R1 = R1 + R3 (loop start)

5221 ; 0A: R2 = R2 + R1

B110 ; 0C: if R1 == R0 jump to 10 (loop exit)

B008 ; 0E: unconditional jump to 08 (R0 == R0 always)

32FF ; 10: screen <- R2 (prints 0F = 15)

B016 ; 12: unconditional jump to 16, skipping the next instruction

33FF ; 14: MUST BE SKIPPED (would print 01)

245C ; 16: R4 = 5C (1.5 in the 8-bit float format)

2569 ; 18: R5 = 69 (2.25)

6645 ; 1A: R6 = R4 + R5 (float) = 6F (3.75)

36FF ; 1C: screen <- R6 (prints 6F)

2748 ; 1E: R7 = 48 (0.5)

2838 ; 20: R8 = 38 (0.25)

6978 ; 22: R9 = R7 + R8 (float) = 4C (0.75)

39FF ; 24: screen <- R9 (prints 4C)

2ADC ; 26: RA = DC (-1.5)

6B5A ; 28: RB = R5 + RA (float) = 69 + DC = 4C (2.25 - 1.5 = 0.75)

3BFF ; 2A: screen <- RB (prints 4C)
```

```
C000 ; 2C: halt
```

Expected result: Screen: `0F 6F 4C 4C`. The instruction at `14` must not execute (no `01` on the screen).

Figure 8: Execution of Test 4.

```
Screen output: .oLL
   (hex: 0F 6F 4C 4C)
PC = 2E   (regs / mem / state show the machine)
vole> state

================ MACHINE STATE ================
--- CPU Registers ---
PC: 2E | IR: C000

R0: 05  R1: 05  R2: 0F  R3: 01
R4: 5C  R5: 69  R6: 6F  R7: 48
R8: 38  R9: 4C  RA: DC  RB: 4C
RC: 00  RD: 00  RE: 00  RF: 00
---------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 20 05 21 00 22 00 23 01 51 13 52 21 B1 10 B0 08
10: 32 FF B0 16 33 FF 24 5C 25 69 66 45 36 FF 27 48
20: 28 38 69 78 39 FF 2A DC 6B 5A 3B FF C0 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:
0F 6F 4C 4C

ASCII: (Invalid ascii or null will be replaced with '.')
.oLL
```

Test 5: Swap Two Registers and Print Them

Purpose: Swap two registers using `MOVE` and print both values.

Program file:

```
None
2105 ; R1 = 05

2206 ; R2 = 06

2300 ; R3 = 00

4013 ; R3 = R1

4021 ; R1 = R2

4032 ; R2 = R3

31FF ; screen R1 -> 06

32FF ; screen R2 -> 05

C000 ; halt
```

Expected result: Screen: `06 05`. Final registers: R1 = `06`, R2 = `05`, R3 = `05`.

Figure 9: Execution of Test 5.

```
=============== MACHINE STATE ===============
--- CPU Registers ---
PC: 12 | IR: C000

R0: 00  R1: 06  R2: 05  R3: 05
R4: 00  R5: 00  R6: 00  R7: 00
R8: 00  R9: 00  RA: 00  RB: 00
RC: 00  RD: 00  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 21 05 22 06 23 00 40 13 40 21 40 32 31 FF 32 FF
10: C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
---------------------
--- Screen Output ---
Hex:
06 05

ASCII: (Invalid ascii or null will be replaced with '.')
..
--------------------
```

Test 6: Integer Addition (Two's Complement)

Purpose: Test two's complement addition with wrap-around and negative numbers.

Program file:

```
None
2180 ; R1 = 80

2280 ; R2 = 80

5312 ; R3 = R1 + R2 = 00 (wrap)

33FF ; screen 00

24FF ; R4 = FF (-1)

2501 ; R5 = 01

5645 ; R6 = R4 + R5 = 00

36FF ; screen 00

2705 ; R7 = 05

28FD ; R8 = FD (-3)

5978 ; R9 = R7 + R8 = 02

39FF ; screen 02

5A88 ; RA = R8 + R8 = FA (-6)

3AFF ; screen FA

C000 ; halt
```

Expected result: Screen: `00 00 02 FA`.

Figure 10: Execution of Test 6.

```
================ MACHINE STATE ================
--- CPU Registers ---
PC: 1E | IR: C000

R0: 00  R1: 80  R2: 80  R3: 00
R4: FF  R5: 01  R6: 00  R7: 05
R8: FD  R9: 02  RA: FA  RB: 00
RC: 00  RD: 00  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 21 80 22 80 53 12 33 FF 24 FF 25 01 56 45 36 FF
10: 27 05 28 FD 59 78 39 FF 5A 88 3A FF C0 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:
00 00 02 FA

ASCII: (Invalid ascii or null will be replaced with '.')
....
--------------------
================================================
```

Test 7: Subtraction Using Two's Complement

Purpose: Compute 9 - 4 and 4 - 9 using XOR and addition (negation by inverting the bits and adding 1).

Program file:

```
None
2109 ; R1 = 09

2204 ; R2 = 04

23FF ; R3 = FF

9432 ; R4 = R3 xor R2 = FB

2501 ; R5 = 01

5445 ; R4 = R4 + R5 = FC (-4)

5614 ; R6 = R1 + R4 = 05

36FF ; screen 05

9731 ; R7 = R3 xor R1 = F6

5875 ; R8 = R7 + R5 = F7 (-9)

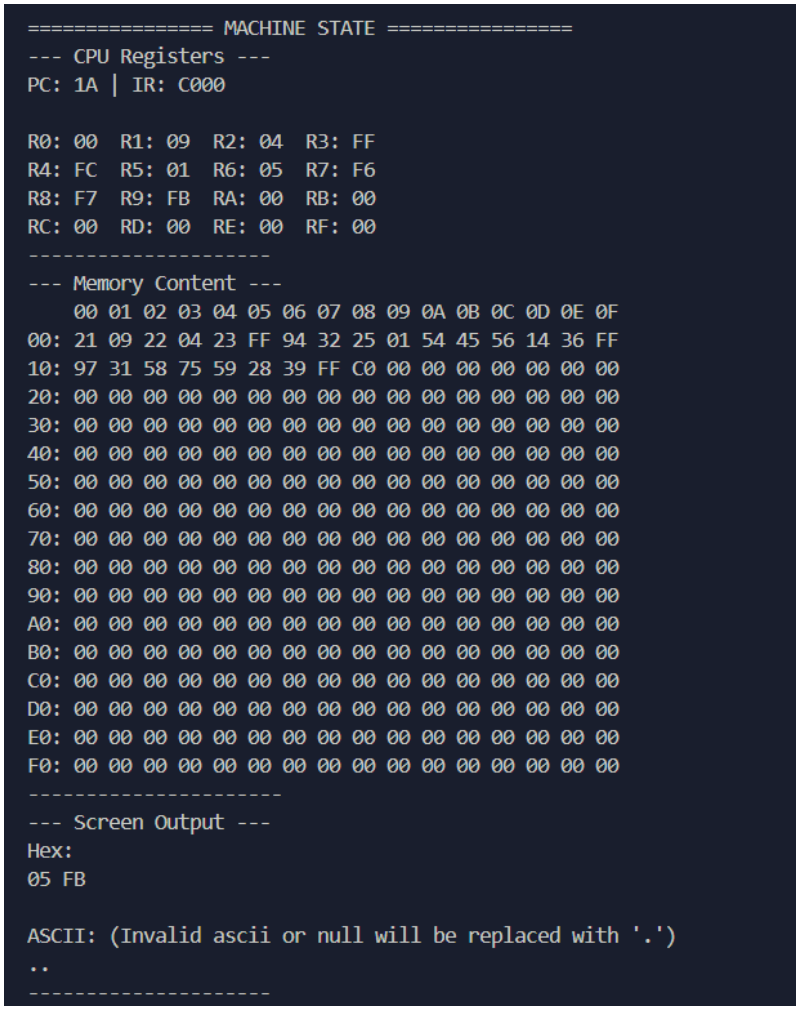
5928 ; R9 = R2 + R8 = FB (4 - 9)

39FF ; screen FB

C000 ; halt
```

Expected result: Screen: `05 FB`.

Figure 11: Execution of Test 7.



Test 8: Bitwise Operations (AND Mask, OR, XOR)

Purpose: Test AND for checking and clearing bits, OR, and XOR.

Program file:

```
202D ; R0 = 2D (00101101)

2104 ; R1 = 04

8201 ; R2 = R0 and R1 = 04
```

```
32FF ; screen 04 (bit 2 is set)

2310 ; R3 = 10

8403 ; R4 = R0 and R3 = 00

34FF ; screen 00 (bit 4 not set)

25F0 ; R5 = F0

260F ; R6 = 0F

7756 ; R7 = R5 or R6 = FF

37FF ; screen FF

28AA ; R8 = AA

29FF ; R9 = FF

9A89 ; RA = R8 xor R9 = 55

3AFF ; screen 55

2B2D ; RB = 2D

2CFB ; RC = FB (mask)

8DBC ; RD = RB and RC = 29 (bit 2 cleared)

3DFF ; screen 29

C000 ; halt
```

Expected result: Screen: `04 00 FF 55 29`.

Figure 12: Execution of Test 8.

```
================ MACHINE STATE ================
--- CPU Registers ---
PC: 28 | IR: C000

R0: 2D  R1: 04  R2: 04  R3: 10
R4: 00  R5: F0  R6: 0F  R7: FF
R8: AA  R9: FF  RA: 55  RB: 2D
RC: FB  RD: 29  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 20 2D 21 04 82 01 32 FF 23 10 84 03 34 FF 25 F0
10: 26 0F 77 56 37 FF 28 AA 29 FF 9A 89 3A FF 2B 2D
20: 2C FB 8D BC 3D FF C0 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:
04 00 FF 55 29

ASCII: (Invalid ascii or null will be replaced with '.')
...U)
--------------------
================================================
```

Test 9: Rotate Right

Purpose: Test rotation by 1, 4, 0, and 8 positions.

Program file:

```
None
2081 ; R0 = 81

A001 ; rotate R0 right by 1 -> C0

30FF ; screen C0

2181 ; R1 = 81

A104 ; rotate right by 4 -> 18

31FF ; screen 18

2281 ; R2 = 81

A200 ; rotate by 0 -> 81

32FF ; screen 81

2381 ; R3 = 81

A308 ; rotate by 8 -> 81

33FF ; screen 81

C000 ; halt
```

Expected result: Screen: `C0 18 81 81`.

Figure 13: Execution of Test 9.

```
================ MACHINE STATE ================
--- CPU Registers ---
PC: 1A | IR: C000

R0: C0  R1: 18  R2: 81  R3: 81
R4: 00  R5: 00  R6: 00  R7: 00
R8: 00  R9: 00  RA: 00  RB: 00
RC: 00  RD: 00  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 20 81 A0 01 30 FF 21 81 A1 04 31 FF 22 81 A2 00
10: 32 FF 23 81 A3 08 33 FF C0 00 00 00 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:
C0 18 81 81

ASCII: (Invalid ascii or null will be replaced with '.')
....
--------------------
================================================
```

Test 10: Conditional Jump Not Taken, Then Taken

Purpose: One jump is not taken, then one is taken and skips two instructions.

Program file:

```
None
2000 ; 00: R0 = 00

2105 ; 02: R1 = 05

B10C ; 04: R1 == R0? no -> not taken

2211 ; 06: R2 = 11

B00E ; 08: R0 == R0 -> always jump to 0E

22EE ; 0A: skipped

22DD ; 0C: skipped

32FF ; 0E: screen 11

C000 ; 10: halt
```

Expected result: Screen: 11. R2 = 11 (not EE or DD).

Figure 14: Execution of Test 10.

```
================ MACHINE STATE ================
--- CPU Registers ---
PC: 12 | IR: C000

R0: 00  R1: 05  R2: 11  R3: 00
R4: 00  R5: 00  R6: 00  R7: 00
R8: 00  R9: 00  RA: 00  RB: 00
RC: 00  RD: 00  RE: 00  RF: 00
--------------------
--- Memory Content ---
    00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00: 20 00 21 05 B1 0C 22 11 B0 0E 22 EE 22 DD 32 FF
10: C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
40: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
F0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
----------------------
--- Screen Output ---
Hex:
11
```

Summary of Test Results

#	Program	Expected Output	Result
1	Swap registers (temp register)	R1=06, R2=05, R3=00	Pass
2	Sum of 1..5 (loop)	0F	Pass
3	Full instruction coverage	0C 02 FC 30 CC 20 0F 3C 99	Pass
4	Loops, jumps, floating point	0F 6F 4C 4C	Pass
5	Swap and print	06 05	Pass
6	Two’s complement addition	00 00 02 FA	Pass
7	Subtraction via two’s complement	05 FB	Pass
8	AND / OR / XOR	04 00 FF 55 29	Pass
9	Rotate right	C0 18 81 81	Pass
10	Conditional jump taken / not taken	11	Pass

Video Link

AI Design Process Video (~10 min):
https://drive.google.com/file/d/1ZTzM-34Vm5GzmK9NhiQBjDic2K8H6YDw/view?usp=drivesdk

GUI Demo Video:
https://drive.google.com/file/d/1imRqRnoSrwbunXqgO0hOCF3LU2qPRao_/view?usp=drivesdk